

MITRE ATT&CK-Based Mini SOC: Attack Simulation, Detection & DFIR Case Creation



Ravintharan Sanjiv
Aspiring Cybersecurity Analyst
July 2026

All attacks were conducted within a controlled laboratory environment for education
and authorized testing purposes only.

Abstract

In this project I have installed a security information and event management system (SIEM) named wazuh and integrated Sysmon as the endpoint telemetry tool to give wazuh more detailed events and also syscheck to configure file integrity monitoring feature in wazuh. Then wazuh agent was used to provision a windows endpoint, together they are used to analyze events and generate alerts to the simulated attacks in the project namely Nmap scan, brute forcing using hydra, PowerShell reverse shell (remote code execution) and file integrity violation. These attacks were mapped to MITRE ATT&CK framework to understand the attack lifecycle and standardize the adversary behavior. Some attacks didn't generate any alert since it can cause alert fatigue to the analyst for how insignificant the attack is in an enterprise level, but for the purpose of this project, I wrote custom detection rules to generate alerts regardless. At last, using DFIR-IRIS case management system, cases were created and assigned to mock soc analysts to imitate a real world security operation center. This project was created to bridge the gap between theoretical cybersecurity knowledge and practical experience. By simulating a real cyber attack detection, analysis and investigation lifecycle.

Table of Content

Introduction.....	7
Project Objectives.....	8
Flow Diagram and System Environment.....	9
Environment Setup.....	10
1. Wazuh Installation.....	10
1.1 Authenticating Wazuh Package.....	10
1.2 Installing Wazuh SIEM.....	10
1.3 Adding Detection Rules.....	12
2. Iris Installation.....	13
3. Registering a Windows machine as a Wazuh agent.....	16
3.1 Deploying Windows host as a Wazuh agent.....	16
3.2 Confirming Windows agent Deployment.....	17
4. Sysmon Installation.....	18
4.1 Installing and Configuring Sysmon.....	18
4.2 Forwarding Sysmon Logs to Wazuh.....	19
5. Configuring Syscheck.....	21
6. Making the Windows machine Vulnerable.....	22
6.1 Enabling SSH service.....	22
6.2 Disabling Windows Security features.....	23
Attack Simulation and Detection.....	24
1. Network Enumeration using Nmap (T1046)	24
1.1 Performing Reconnaissance [TA0043].....	24
1.2 Wazuh Detection.....	24
2. Brute Forcing using Hydra (T1110.001)	25
2.1 Creating a Password Dictionary.....	25
2.2 Credential Guessing (Initial access) [TA0001].....	26
2.3 Wazuh Detection.....	26
3. Establishing a PowerShell reverse shell (T1059.001).....	27
3.1 C2 Listener Setup.....	27

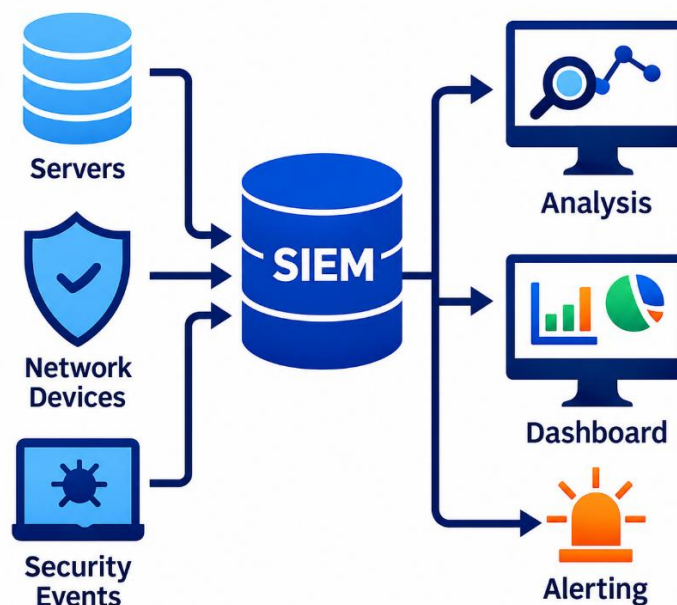
3.2	Executing the PowerShell Reverse shell Payload.....	27
3.3	Establishing C2(Command and Control) [TA0011].....	28
3.4	Wazuh Detection.....	28
4.	File Integrity Violation (T1565.001), (T1222.001)	29
4.1	Impact Stage of the Attack chain [TA0040].....	29
4.2	Wazuh Detection.....	29
Iris Case Creation.....		30
1.	Case Creation for Hydra Brute Force.....	30
2.	Case Creation for PowerShell Reverse shell.....	31
3.	Case Creation for File Integrity Violation.....	33
Conclusion.....		34

Table of Figures

Figure 1: Verifying the authenticity of Wazuh packages.....	10
Figure 2: Quick Installation of Wazuh.....	10
Figure 3: Wazuh login page.....	11
Figure 4: Wazuh dashboard.....	11
Figure 5: Detection rule for Nmap scans.....	12
Figure 6: Detection rule for ssh brute force attack.....	12
Figure 7: Detection rule validation and service restart.....	13
Figure 8: Clonning Iris from github.....	13
Figure 9: Downloading Iris components.....	14
Figure 10: Installing Iris.....	14
Figure 11: Iris login page.....	15
Figure 12: Iris dashboard.....	15
Figure 13: Adding Windows as an agent in Wazuh (1)	16
Figure 14: Adding Windows as an agent in Wazuh (2)	16
Figure 15: Configuring Windows as an Wazuh agent in Windows.....	17
Figure 16: Confirming the agent deployment from the Wazuh server.....	17
Figure 17: Confirming the agent deployment from the Wazuh webapp.....	18
Figure 18: Preinstalled Sysmon files.....	18
Figure 19: Installation and configuration of Sysmon.....	19
Figure 20: Forwarding Sysmon logs to Wazuh manager.....	19
Figure 21: Restarting the Wazuh service to save changes.....	20
Figure 22: Confirming wazuh recieves the sysmon logs.....	20
Figure 23: The folder that will be monitored.....	21
Figure 24: Establishing the folder to be monitored.....	21
Figure 25: Confirmation of wazuh FIM.....	22
Figure 26: Installing SSH.....	22
Figure 27: Starting the SSH service.....	23
Figure 28: Disabling Windows default Security features.....	23

Figure 29: Nmaping to windows machine.....	24
Figure 30: Alert generated for Nmap (1)	24
Figure 31: Alert generated for Nmap (2)	25
Figure 32: Creating a wordlist.....	25
Figure 33: Brute Forcing using Hydra.....	26
Figure 34: Alert generated for failed SSH.....	26
Figure 35: Establishing a listener for the reverse shell.....	27
Figure 36: Executing the PowerShell based reverse TCP shell code.....	27
Figure 37: C2 Establishment.....	28
Figure 38: Alert generated for PowerShell Reverse shell.....	28
Figure 39: Stored data & permission modification.....	29
Figure 40: Alert generated for stored data manipulation.....	29
Figure 41: Creating a case for the Brute Force attack.....	30
Figure 42: Adding the extracted information from wazuh.....	30
Figure 43: Successfully created a IRIS case for the Brute Force attack.....	31
Figure 44: Creating a case for powershell reverse shell.....	31
Figure 45: Adding the extracted information from wazuh.....	32
Figure 46: Successfully created a IRIS case for the powershell reverse shell.....	32
Figure 47: Adding the extracted information from wazuh.....	33
Figure 48: Successfully created a IRIS case for file integrity violation.....	33

Introduction



Modern businesses are struggling to detect cyber attacks because each system must be checked separately to see a full attack since these events are individual and looks independent inducing a major attack could go unnoticed without centralized correlation of events. The enormous number of logs generated can also be time consuming and exhausting to analyze making all these problems contribute to high risk of missing out real attacks. This is why a SIEM (Security Information Management System) is used and for this project I have implemented wazuh a open source security monitoring tool that provides capabilities associated with SIEM and XDR(Extended Detection and Response) designed to collect, analyze and monitor endpoints, servers, network devices and other infrastructure components for suspicious activity. But it is also important to manage the security events among the security teams, investigate and document the incidents and for this purpose DFIR-IRIS acts as the centralized workplace platform.

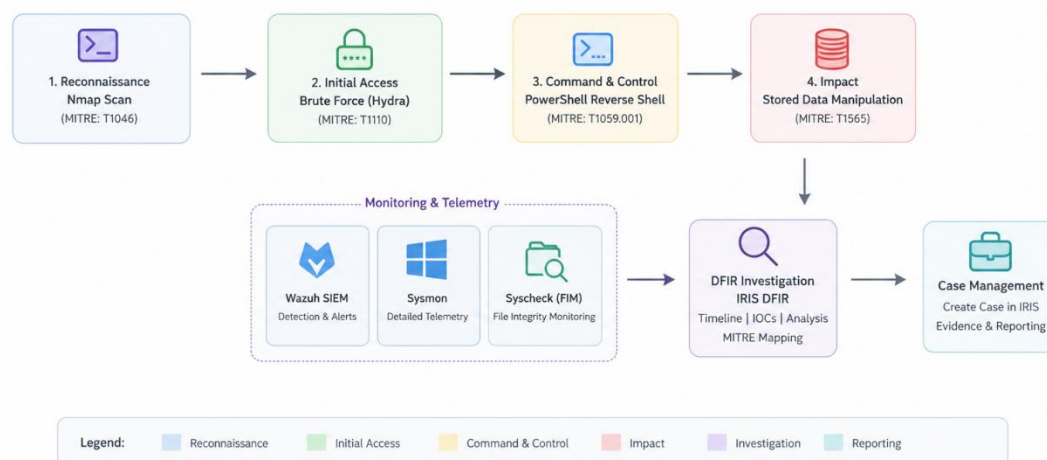
However, for aspiring security professional learning concepts of log analysis, threat detection, incident response and malware analysis can only get them so far and its challenging to get the opportunity or the access to learn in realistic SOC environments. For this reason I have built a mini SOC laboratory environment that consist of real world attacks and investigations using industry relevant security tools such as wazuh and IRIS to gain a practical SOC experience.

In this project I have simulated MITRE ATT&CK mapped attacks like Nmap scan, brute forcing, PowerShell reverse shell and stored data manipulation. Then these attacks were monitored using wazuh and alerts were generated according to the custom detection rules. Lastly cases were created for each event using IRIS for centralized management and investigation.

Project Objective

- Deploying Wazuh management stack on the server
- Installing DFIR-IRIS case management system
- Registering a host using Wazuh agent
- Sysmon and syscheck integration with Wazuh
- Writing custom detection rules for wazuh
- Simulating attacks and mapping them to MITRE ATT&CK framework
- Detecting the simulated attacks using Wazuh
- Case creation in IRIS

Flow Diagram and System Environment



Role	Operating system	Hypervisor	Network Adapter
Host Machine	Windows 11 Home 25H2	N/A	Bridged Network
Victim Machine	Windows 10 Home 21H2	VMware Workstation 17 Pro 17.5.2	
Attacker Machine	Kali GNU/Linux Rolling 2025.4		
Hosting Wazuh SIEM	Ubuntu 22.04.5 LTS		
Hosting Iris DFIR Platform			

Environment Setup

1. Wazuh Installation

1.1 Authenticating Wazuh Package

`curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg -- dearmor -o /usr/share/keyrings/wazuh-archive-keyring.gpg`



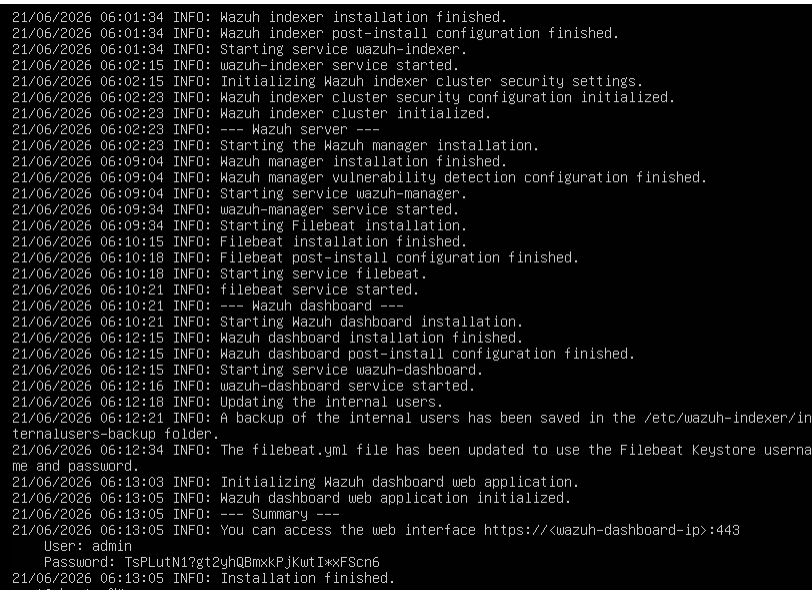
```
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --dearmor -o /usr/share/keyrings/wazuh-archive-keyring.gpg
```

Figure 1: Verifying the authenticity of Wazuh packages

The first half of the command downloads the Wazuh's public key from the official wazuh package server and the second half converts the ASCII armored public key into binary format, which the APT can use later. This process is done to verify whether the package we are downloading is authentic and not tampered in between or before the download.

1.2 Installing Wazuh SIEM

`curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a`



```
21/06/2026 06:01:34 INFO: Wazuh indexer installation finished.
21/06/2026 06:01:34 INFO: Wazuh indexer post-install configuration finished.
21/06/2026 06:01:34 INFO: Starting service wazuh-indexer.
21/06/2026 06:02:15 INFO: wazuh-indexer service started.
21/06/2026 06:02:15 INFO: Initializing Wazuh indexer cluster security settings.
21/06/2026 06:02:23 INFO: Wazuh indexer cluster security configuration initialized.
21/06/2026 06:02:23 INFO: Wazuh indexer cluster initialized.
21/06/2026 06:02:23 INFO: --- Wazuh server ---
21/06/2026 06:02:23 INFO: Starting the Wazuh manager installation.
21/06/2026 06:09:04 INFO: Wazuh manager installation finished.
21/06/2026 06:09:04 INFO: Wazuh manager vulnerability detection configuration finished.
21/06/2026 06:09:04 INFO: Starting service wazuh-manager.
21/06/2026 06:09:34 INFO: wazuh-manager service started.
21/06/2026 06:09:34 INFO: Starting Filebeat installation.
21/06/2026 06:10:15 INFO: Filebeat installation finished.
21/06/2026 06:10:18 INFO: Filebeat post-install configuration finished.
21/06/2026 06:10:18 INFO: Starting service filebeat.
21/06/2026 06:10:21 INFO: filebeat service started.
21/06/2026 06:10:21 INFO: --- Wazuh dashboard ---
21/06/2026 06:10:21 INFO: Starting Wazuh dashboard installation.
21/06/2026 06:12:15 INFO: Wazuh dashboard installation finished.
21/06/2026 06:12:15 INFO: Wazuh dashboard post-install configuration finished.
21/06/2026 06:12:15 INFO: Starting service wazuh-dashboard.
21/06/2026 06:12:16 INFO: wazuh-dashboard service started.
21/06/2026 06:12:18 INFO: Updating the internal users.
21/06/2026 06:12:21 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.
21/06/2026 06:12:34 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and password.
21/06/2026 06:13:03 INFO: Initializing Wazuh dashboard web application.
21/06/2026 06:13:05 INFO: Wazuh dashboard web application initialized.
21/06/2026 06:13:05 INFO: --- Summary ---
21/06/2026 06:13:05 INFO: You can access the web interface https://wazuh-dashboard-ip:443
User: admin
Password: TsPLutN1?gt2yhQBMxkPJkwtI*xFSn6
21/06/2026 06:13:05 INFO: Installation finished.
root@ubuntu:~#
```

Figure 2: Quick Installation of Wazuh

Instead of manually installing wazuh components, I used the Quick Installation command. After successfully installing the package, I got my admin account password and the port number that is used to access the SIEM.

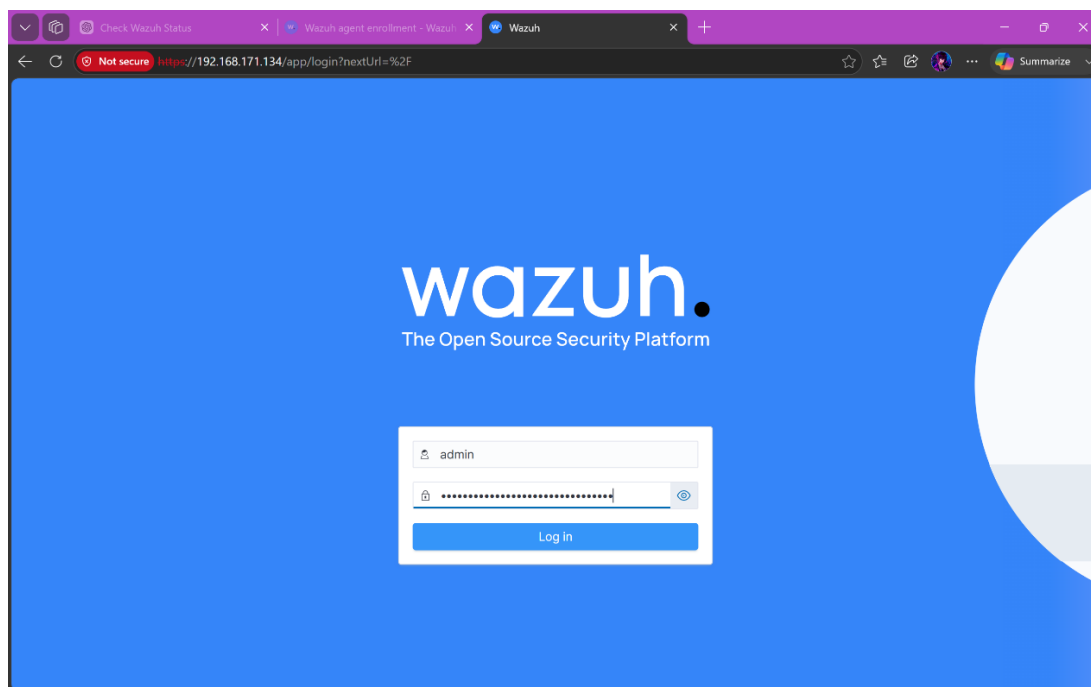


Figure 3: Wazuh login page

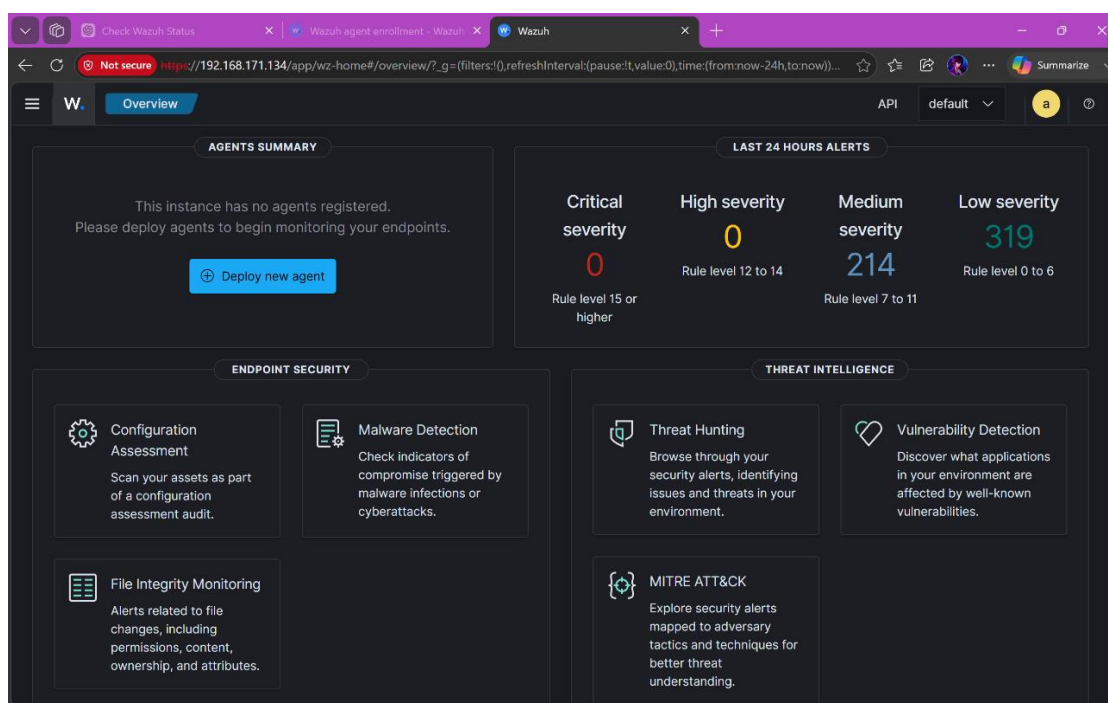


Figure 4: Wazuh dashboard

Successfully installed Wazuh SIEM and enabled dark mode. The SIEM was then accessed from the Windows host machine using the Ubuntu sever's IP address.

1.3 Adding Detection Rules

`"/var/ossec/etc/rules/local_rules.xml"`

```
<group name="custom,nmap,">
  <rule id="100200" level="10" frequency="20" timeframe="30">
    <if_matched_sid>61605</if_matched_sid>
  </if_matched_sid>source_ip/>
  <description>Possible Nmap scan detected.</description>
  <mitre>
    <id>T1046</id>
  </mitre>
</rule>
</group>
```

Figure 5: Detection rule for Nmap scans

Since wazuh isn't a network intrusion detection system in default it doesn't log network activities, therefore I made the wazuh agent in the windows to collect the sysmon event logs and created a custom rule to generate an alert if the same source ip generates 20 sysmon network connection events within 30 seconds.

```
<rule id="5716" level="5">
  <srcip>1.1.1.1</srcip>
  <description>sshd: authentication failed from IP 1.1.1.1.</description>
  <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>

<!-- single failed login -->

<group name="windows,authentication,">
  <rule id="100100" level="5">
    <if_sid>60122</if_sid>
    <field name="win.system.eventID">4625</field>
    <description>Windows failed login detected.</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <!-- Brute-force -->
  <rule id="100101" level="10" frequency="5" timeframe="60">
    <if_matched_sid>100100</if_matched_sid>
    <same_source_ip>
    <description>Possible SSH brute-force attack against Windows host.</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>
</group>
```

Figure 6: Detection rule for ssh brute force attack

This is a two stage detection and correlation rule, the first rule detects a single failed login, and then the second rule detects the brute force. This custom rule gets triggered if the 1st rule gets triggered 5 times within 60 seconds from the same ip and tags it to the MITRE ATT&CK mapping T110.

```

~
~
~
~
"/var/ossec/etc/rules/local_rules.xml" 60L, 1413B written
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# sudo /var/ossec/bin/wazuh-analysisd -t
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# systemctl restart wazuh-manager.service
root@ubuntu:~# _

```

Figure 7: Detection rule validation and service restart

`sudo /var/ossec/bin/wazuh-analysisd -t`

`systemctl restart wazuh-manager.service`

The added Detection rules are then validated using the above command and then the wazuh service is restarted.

2. Iris Installation

`docker-compose --version`

`git clone https://github.com/dfir-iris/iris-web.git`

```

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
systemctl restart user@1000.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# sudo systemctl enable docker
root@ubuntu:~# sudo systemctl start docker
root@ubuntu:~# docker-compose --version
docker-compose version 1.29.2, build unknown
root@ubuntu:~# git clone https://github.com/dfir-iris/iris-web.git
Cloning into 'iris-web'...
remote: Enumerating objects: 59963, done.
remote: Counting objects: 100% (1488/1488), done.
remote: Compressing objects: 100% (257/257), done.
remote: Total 59963 (delta 1328), reused 1231 (delta 1231), pack-reused 58475 (from 3)
Receiving objects: 100% (59963/59963), 35.08 MiB | 2.00 MiB/s, done.
Resolving deltas: 100% (47740/47740), done.
root@ubuntu:~# _

```

Figure 8: Clonning Iris from github

I previously installed docker since Iris is a containerized application and then I cloned the Iris Project from the gitub , this contains the blueprints of how to build the Iris webapp, it consists of wich container to create, what ports to expose, which image the container should use and etc.

`docker compose pull`

```

stats          Display a live stream of container(s) resource usage statistics
stop           Stop services
top            Display the running processes
unpause        Unpause services
up             Create and start containers
version        Show the Docker Compose version information
volumes        List volumes
wait           Block until containers of all (or specified) services stop.
watch          Watch build context for service and rebuild/refresh containers when files
are updated

Run 'docker compose COMMAND --help' for more information on a command.
root@ubuntu:~/iris-web# docker compose pull
[+] pull 42/60
* Image ghcr.io/dfir-iris/iriswebapp... [*****] 44.12MB / 558.1MB Pulling 132.2s
* Image ghcr.io/dfir-iris/iriswebapp... [*****] 13.68MB / 109.2MB Pulling 132.2s
* Image rabbitmq:3-management-alpine Pulling 132.2s
* Image ghcr.io/dfir-iris/iriswebapp_db:latest [*****] 7.34MB / 106.1MB Pulling 132.2s
failed to copy: httpReadSeeker: failed open: failed to do request: Get "https://registry-1.docker.io
/v2/library/rabbitmq/manifests/sha256:3c498e636fd64462480c5f9ff842eb224ab84160a8ada1ded5375e9569e923
0c": dial tcp 54.173.59.97:443: i/o timeout
root@ubuntu:~/iris-web# docker compose pull
[+] pull 1/57
* Image ghcr.io/dfir-iris/iriswebapp_db:latest [*****] Pulling 22.0s
* Image ghcr.io/dfir-iris/iriswebapp_nginx:latest [*****] Pulling 22.0s
* Image ghcr.io/dfir-iris/iriswebapp_app:latest [*****] Pulling 22.0s
* Image rabbitmq:3-management-alpine Error failed ... 22.0s
Error response from daemon: failed to copy: httpReadSeeker: failed open: failed to do request: Get "
https://registry-1.docker.io/v2/library/rabbitmq/manifests/sha256:606d8c0d6b3c18d1da9afc53bc7c0b2a8d
5486df91b5a9830e9e07626c9ae281": net/http: TLS handshake timeout
root@ubuntu:~/iris-web# docker compose pull
[+] pull 16/69
* Image ghcr.io/dfir-iris/iriswebapp_app:latest Pulled 378.3s
* Image ghcr.io/dfir-iris/iriswebapp_nginx:latest Pulled 166.8s
* Image rabbitmq:3-management-alpine Pulled 370.7s
* Image ghcr.io/dfir-iris/iriswebapp_db:latest Pulled 290.3s
root@ubuntu:~/iris-web# _

```

Figure 9: Downloading Iris components

Now the docker opens the docker-compose.yml file, which was installed previously from github. Now that it knows the instructions of like, what image to use and how containers should communicate, docker proceeds with installing them from docker hub or github container registry.

docker compose up

```

2026-06-21 12:55:05 :: INFO :: module_handler :: dup_logs :: Module type : module_processor
2026-06-21 12:55:05 :: INFO :: module_handler :: dup_logs :: Module health validated
2026-06-21 12:55:05 :: INFO :: module_handler :: register_module :: Parsing configuration
2026-06-21 12:55:05 :: INFO :: module_handler :: register_module :: Adding module
2026-06-21 12:55:05 :: INFO :: module_handler :: deregister_from_hook :: Deregistering module #5 fro
m on_postload_ioc_create
2026-06-21 12:55:05 :: INFO :: module_handler :: deregister_from_hook :: Deregistering module #5 fro
m on_postload_ioc_update
2026-06-21 12:55:05 :: INFO :: module_handler :: deregister_from_hook :: Deregistering module #5 fro
m on_manual_trigger_ioc
2026-06-21 12:55:05 :: INFO :: post_init :: register_default_modules :: Successfully registered iris
_intelowl_module
2026-06-21 12:55:05 :: INFO :: post_init :: run_post_init :: Creating initial customer
2026-06-21 12:55:05 :: INFO :: post_init :: run_post_init :: Creating initial case
2026-06-21 12:55:05 :: INFO :: post_init :: run_post_init :: Creating symlinks for custom asset icon
s
2026-06-21 12:55:05 :: INFO :: post_init :: run_post_init :: Post-init steps completed
2026-06-21 12:55:05 :: WARNING :: post_init :: run_post_init :: =====
2026-06-21 12:55:05 :: WARNING :: post_init :: run_post_init :: | IRIS IS READY on port 443 |
2026-06-21 12:55:05 :: WARNING :: post_init :: run_post_init :: =====
2026-06-21 12:55:05 :: INFO :: post_init :: run_post_init :: You can now login with user administrat
or and password >>> =5V$sUQD(>,kq)Lr <<< on 443
2026-06-21 12:59:59 :: INFO :: tracker :: track_activity :: Anonymous :: Case None :: Wrong login pa
ssword for user 'administrator' using local auth
2026-06-21 13:00:54 :: INFO :: tracker :: track_activity :: Anonymous :: Case None :: Wrong login pa
ssword for user 'administrator' using local auth

```

Figure 10: Installing Iris

Now the docker starts assembling the images and other components and starts running containers. Once its finished it states the port number the tool should be accessed and the password for admin access.

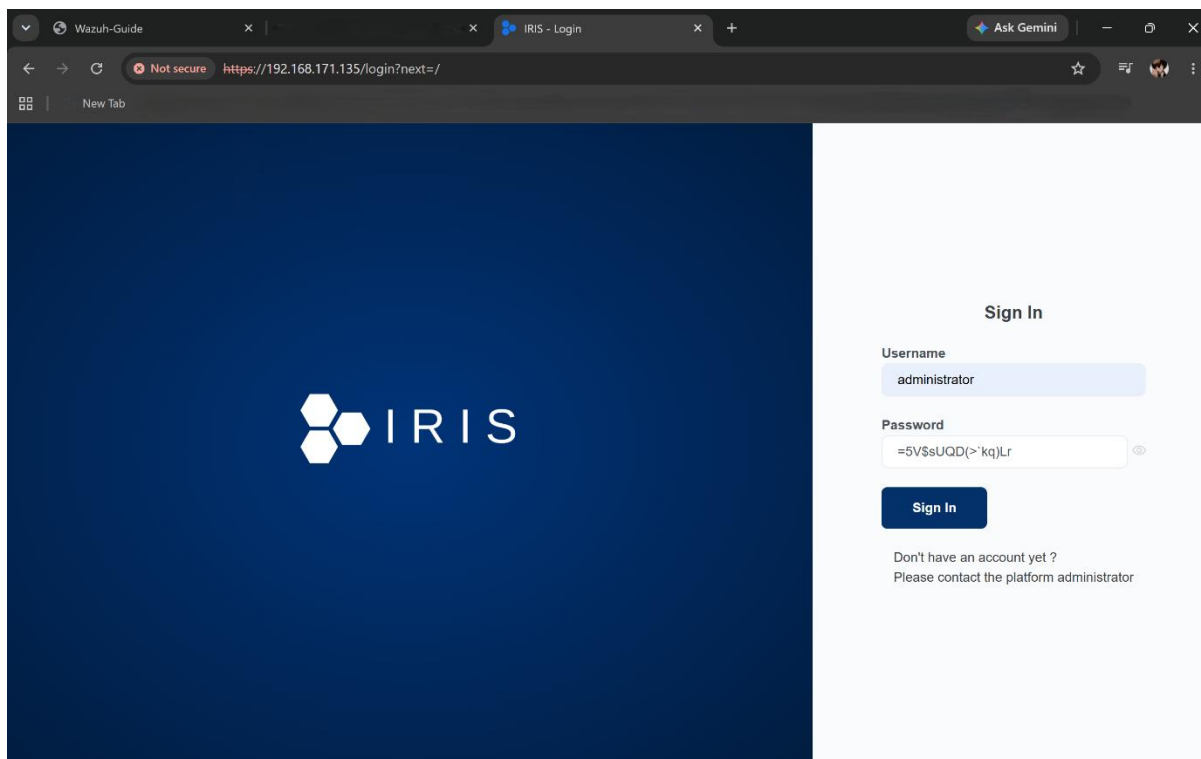


Figure 11: Iris login page

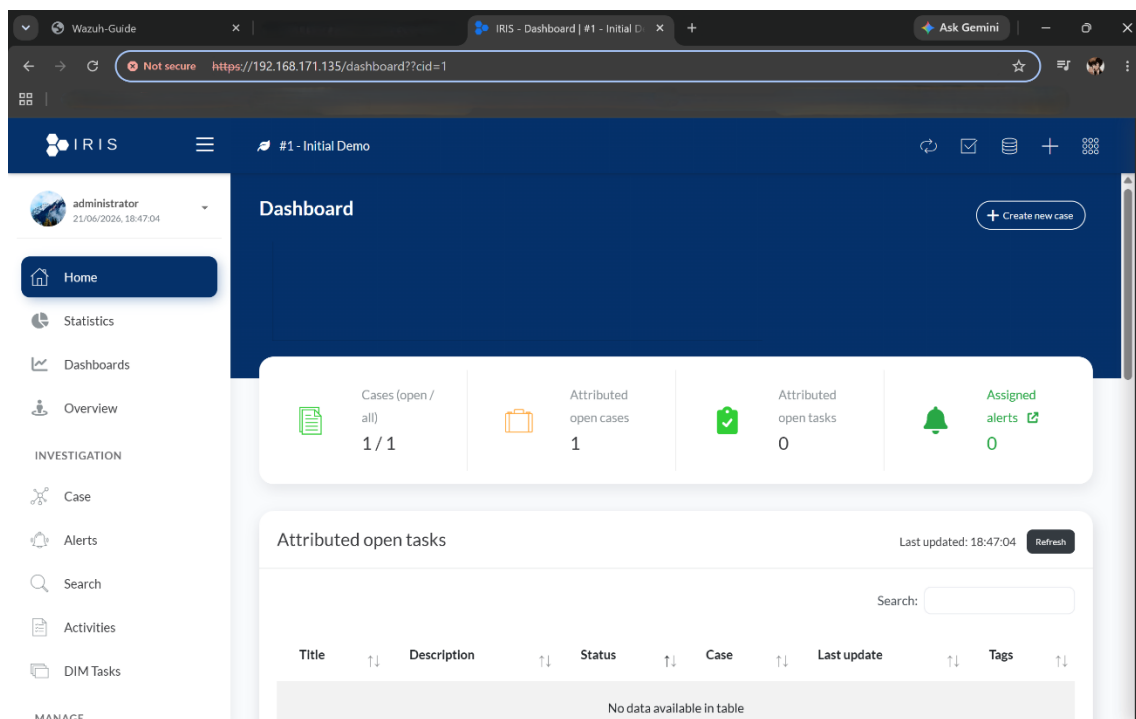


Figure 12: Iris dashboard

3. Registering the Windows machine as a Wazuh agent

3.1 Deploying Windows host as a Wazuh agent

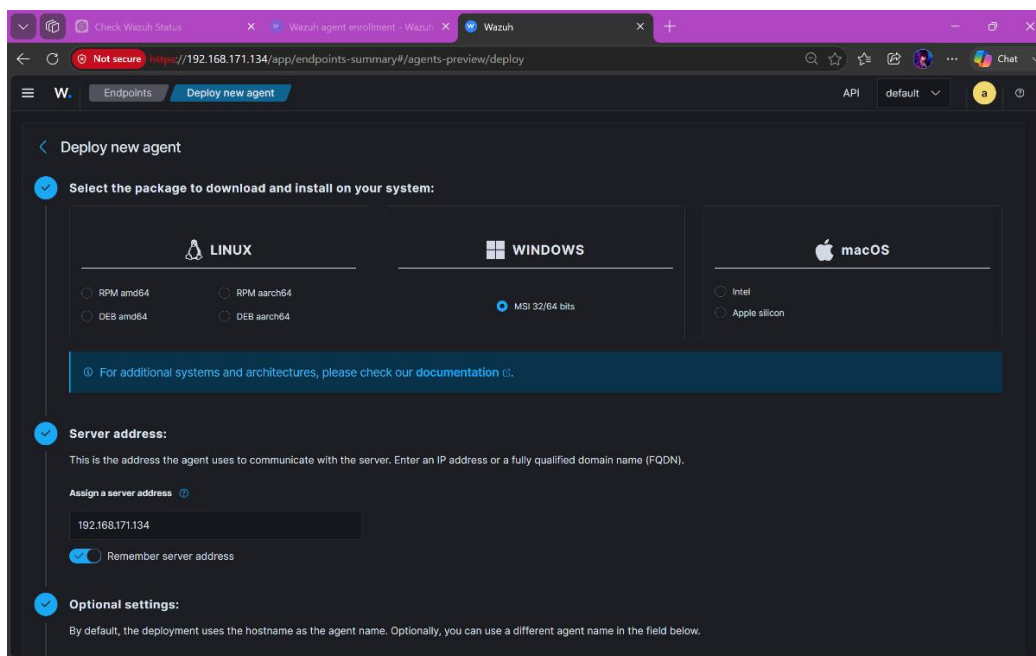


Figure 13: Adding Windows as an agent in Wazuh (1)

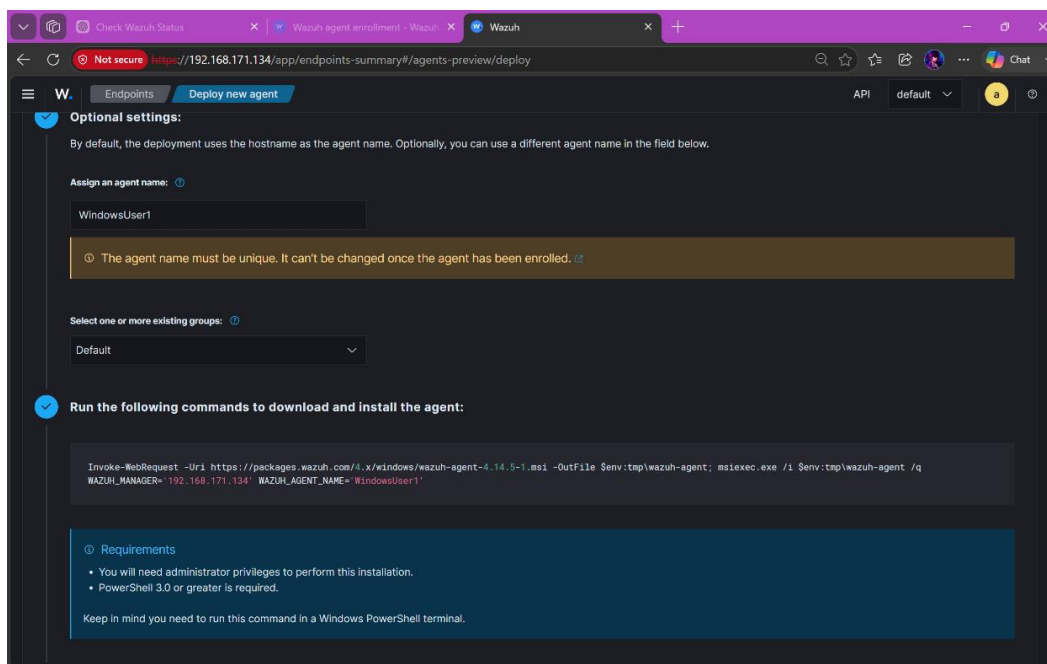


Figure 14: Adding Windows as an agent in Wazuh (2)

Here I deployed configured it by assigning the Wazuh's IP as the server address and agent name as WindowsUser1.

NET START Wazuh

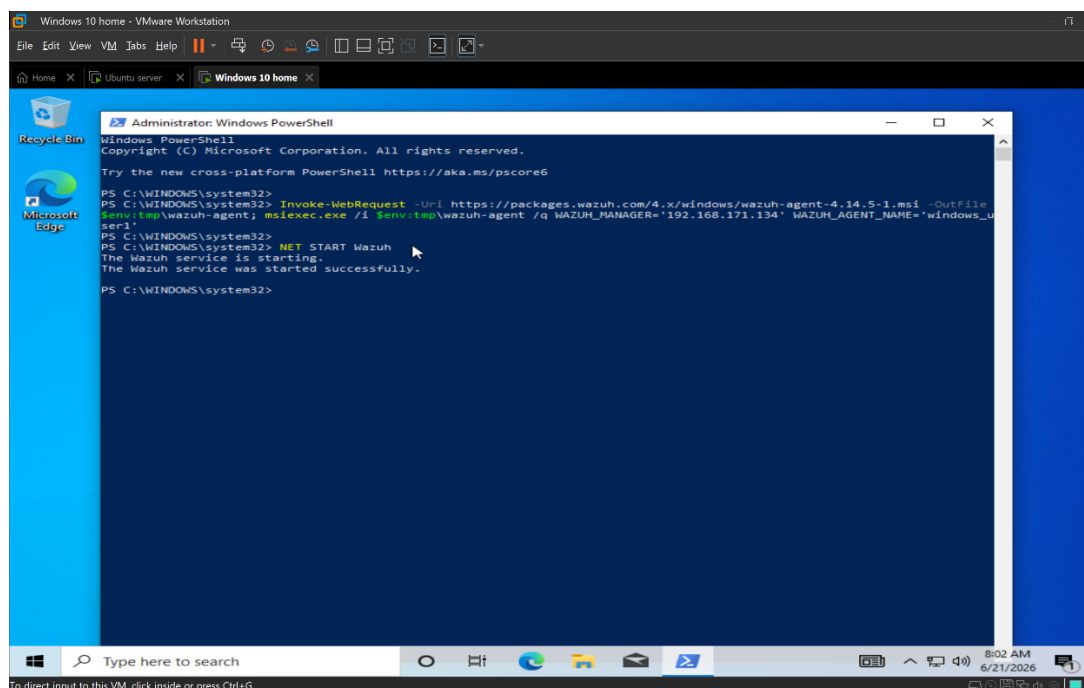


Figure 15: Configuring Windows as an Wazuh agent in Windows

After pasting the command that was given by the wazuh, I started the Wazuh service.

3.2 Confirming Windows agent Deployment

`sudo /var/ossec/bin/agent_control -l`

```

lines 1-23/23 (END)
root@ubuntu:~# systemctl start wazuh-manager.service
root@ubuntu:~# systemctl status wazuh-dashboard.service
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-06-23 14:44:52 UTC; 38min ago
     Main PID: 900 (node)
        Tasks: 11 (limit: 4510)
       Memory: 345.9M
          CPU: 42.516s
      CGroup: /system.slice/wazuh-dashboard.service
              └─900 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536

Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"error","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"log","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"error","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"error","timestamp":"2026-06-23T15:20:03.000Z"}
Jun 23 15:20:03 ubuntu opensearch-dashboards[900]: {"type":"response","timestamp":"2026-06-23T15:20:03.000Z"}
lines 1-20/20 (END)

root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: ubuntu (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: windows_user1, IP: any, Active

List of agentless devices:

root@ubuntu:~#
  
```

Figure 16: Confirming the agent deployment from the Wazuh server

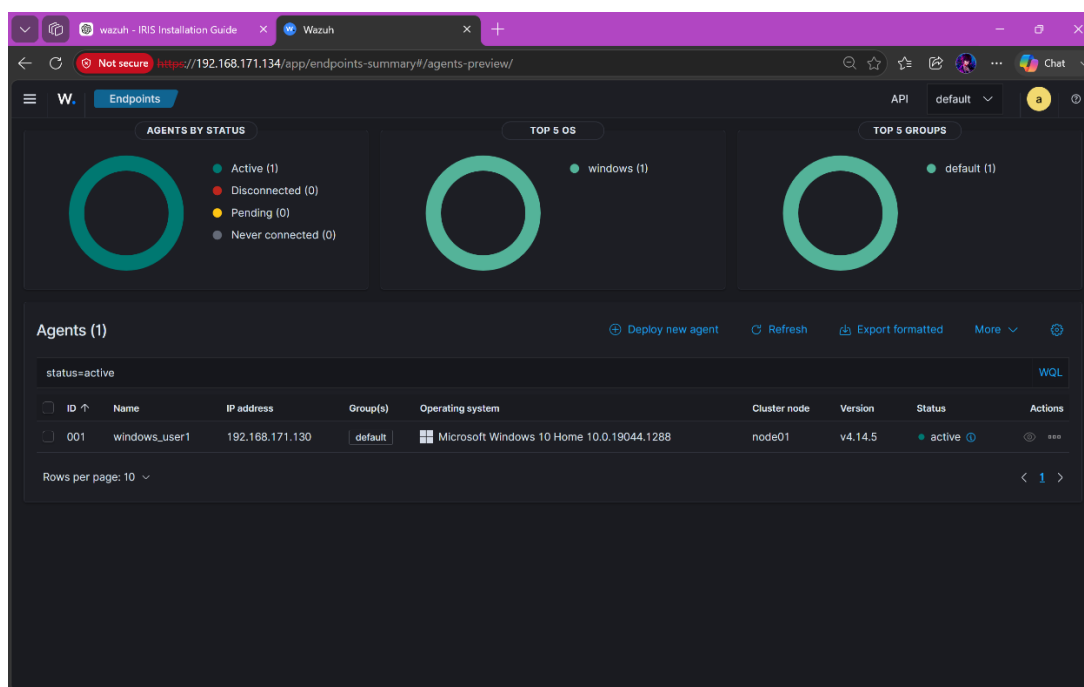


Figure 17: Confirming the agent deployment from the Wazuh webapp

This confirms that, we have successfully deployed the Windows agent. In the Wazuh webapp, the windows ip address, Windows version and the status is visible.

4. Sysmon Installation

4.1 Installing and Configuring Sysmon

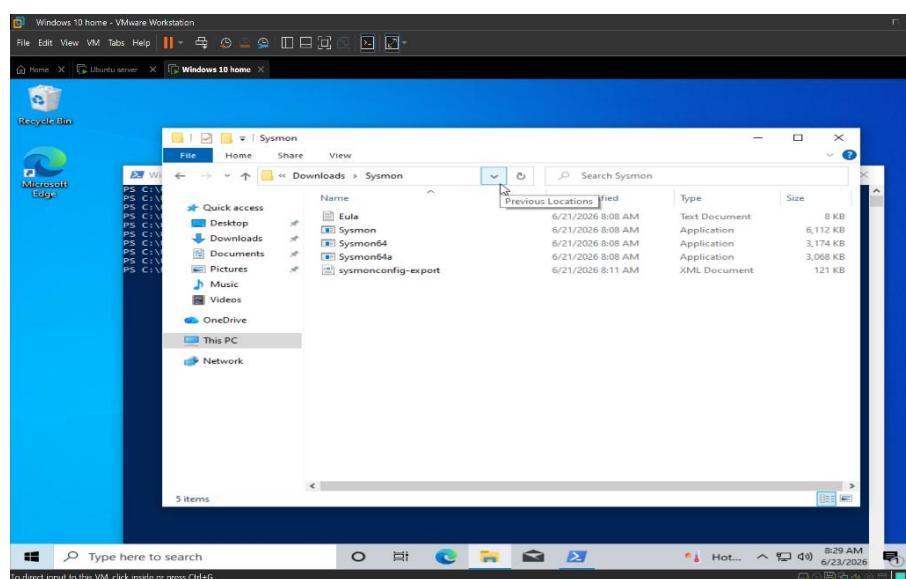


Figure 18: Preinstalled Sysmon files

```
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

```
Get-service Sysmon64
```

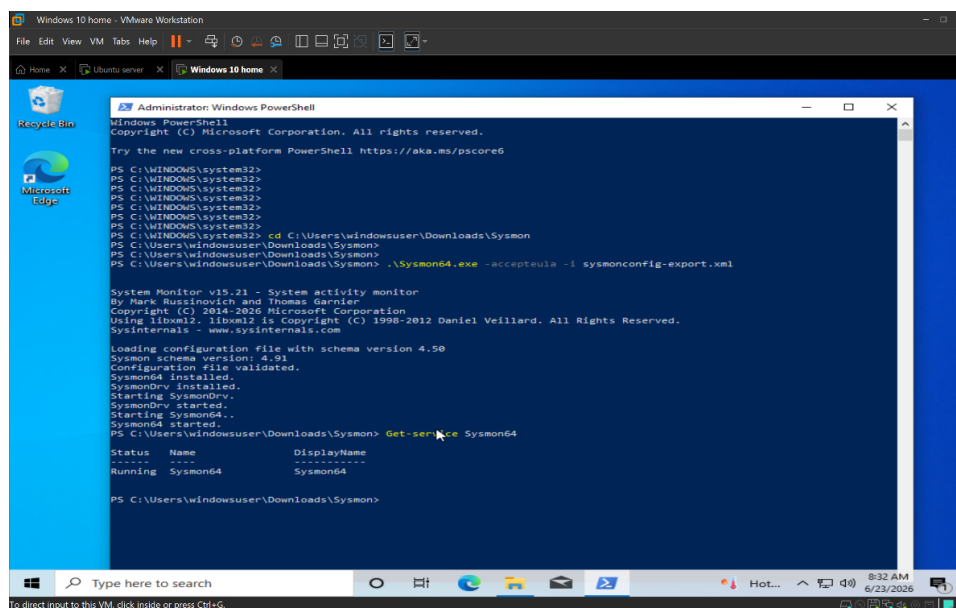


Figure 19: Installation and configuration of Sysmon

This command installs Sysmon as a Windows service and loads the `sysmonconfig-export.xml` configuration file that has the instructions on what the Sysmon has to monitor and then the second command is used to check the status of the service.

4.2 Forwarding Sysmon Logs to Wazuh

`C:\Program Files (x86)\ossec-agent\ossec.conf`

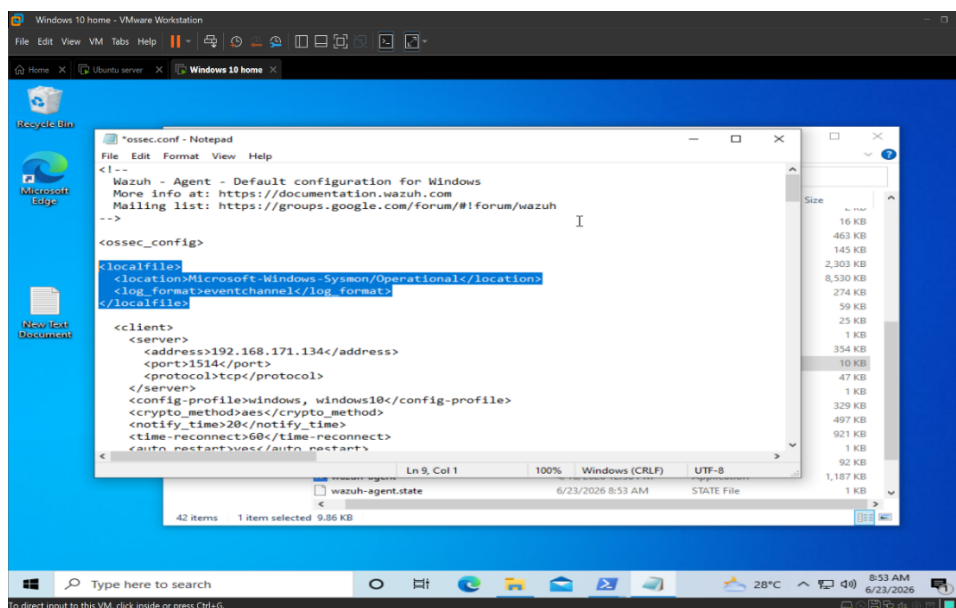


Figure 20: Forwarding Sysmon logs to Wazuh manager

This edit in the `ossec.conf` file is done to forward Sysmon logs from the Windows event log to Wazuh manager for analysis. Basically the wazuh agent says to monitor Sysmon operational event log which contains process creation, Registry changes and etc. It also specifies the source of Windows event channel so Wazuh knows to use the Windows event log API to extract information instead of reading a text log file.

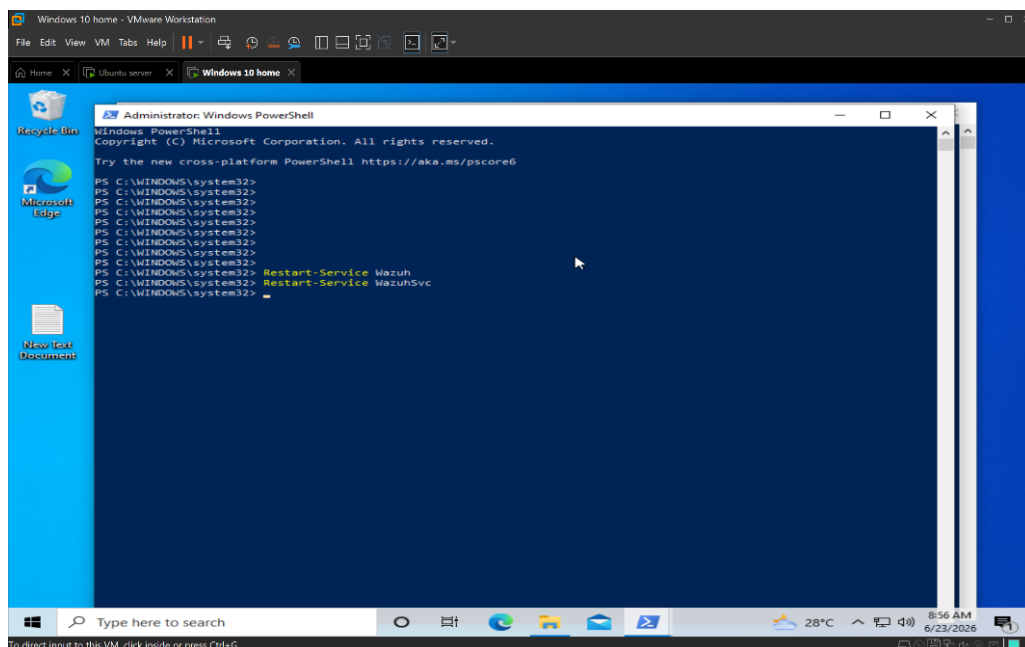


Figure 21: Restarting the Wazuh service to save changes

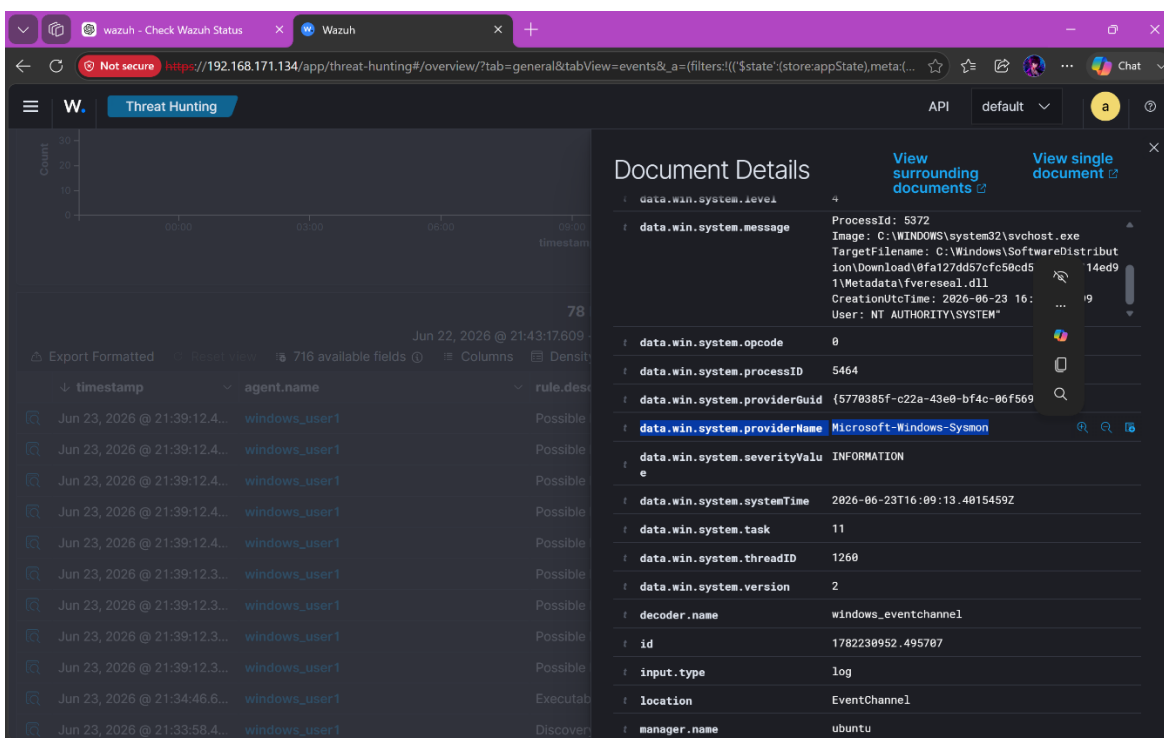


Figure 22: Confirming wazuh receives the sysmon logs

5. Configuring Syscheck

C:\User1\Sensitive Data

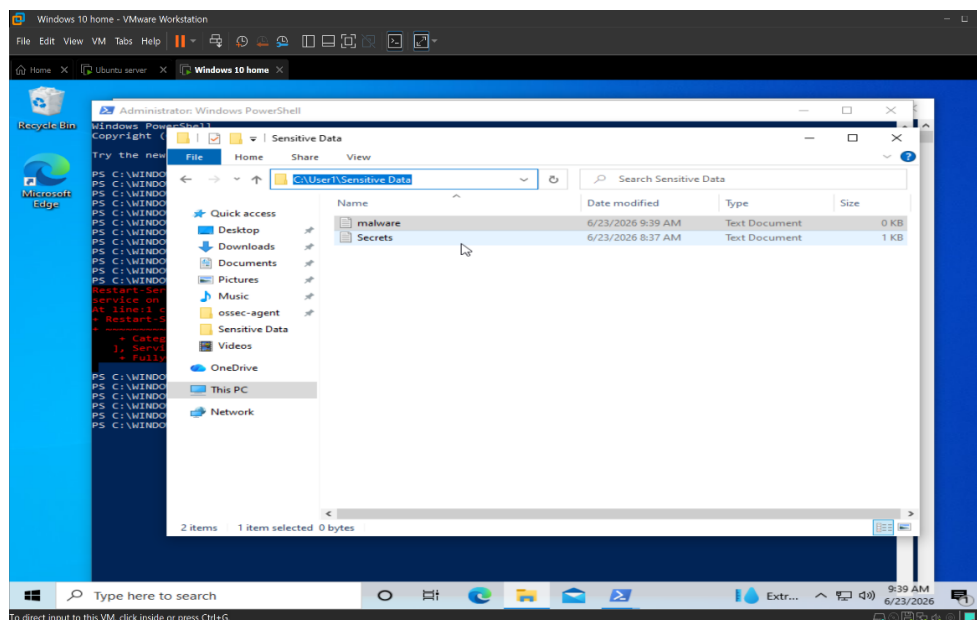


Figure 23: The folder that will be monitored

```
<directories realtime="yes">C: \User1\Sensitive Data</directories>
```

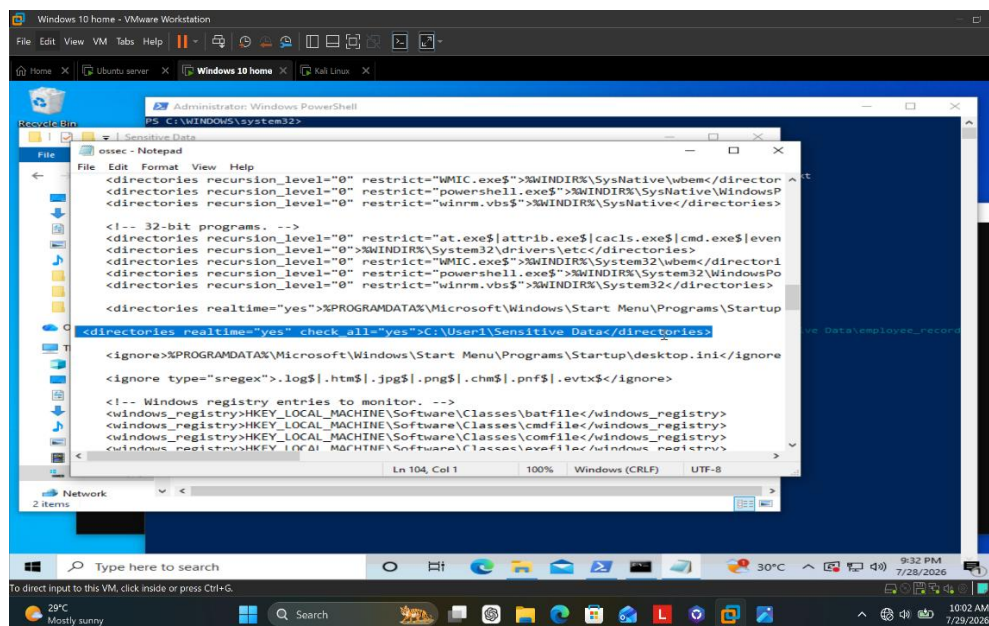


Figure 24: Establishing the folder to be monitored

The above line, which has the folder path to be monitored is added under the file integrity monitoring section inside the same C:\Program Files (x86)\ossec-agent\ossec.conf file.

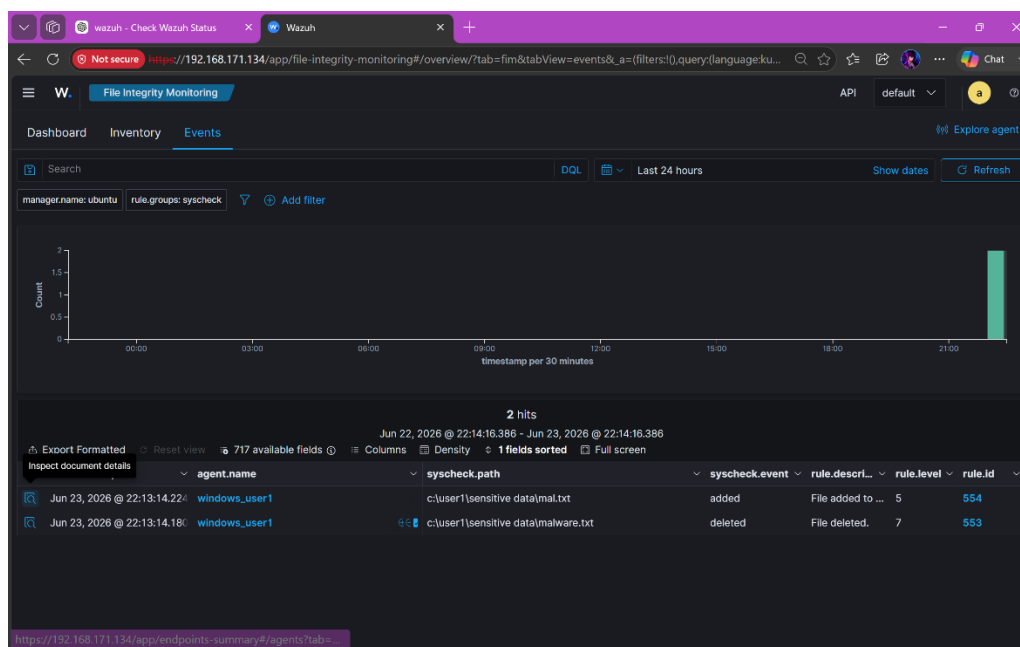


Figure 25: Confirmation of wazuh FIM

6. Making the Windows machine Vulnerable

6.1 Enabling SSH service

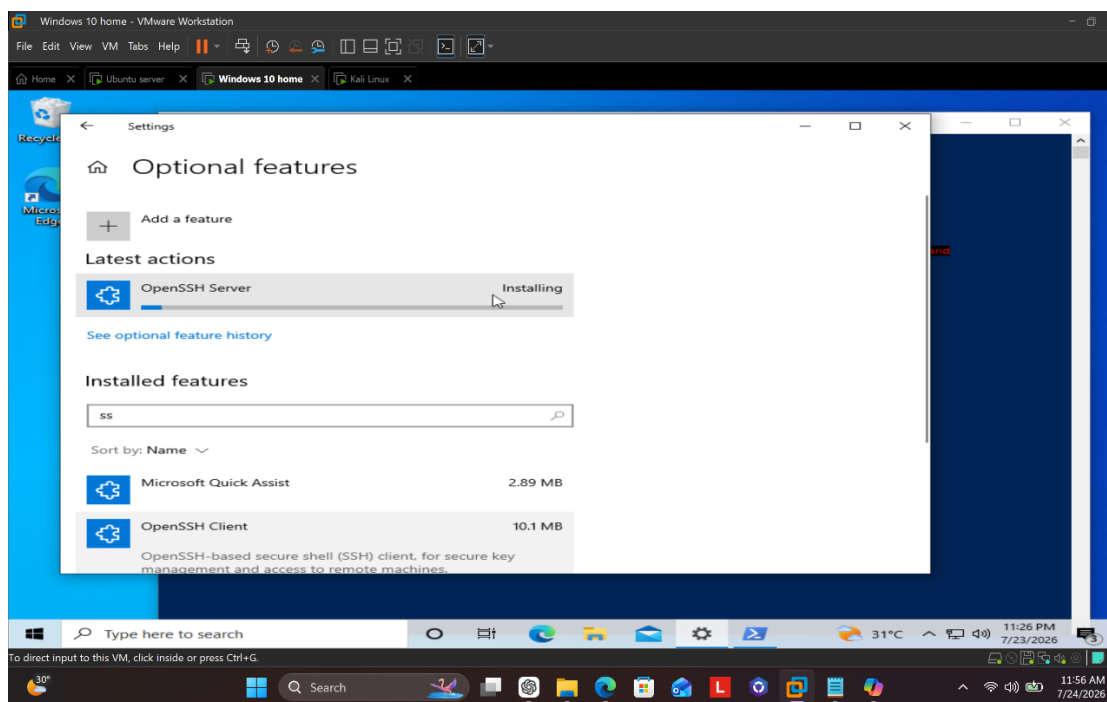


Figure 26: Installing SSH

START-Service sshd

GET-SERVICE sshd

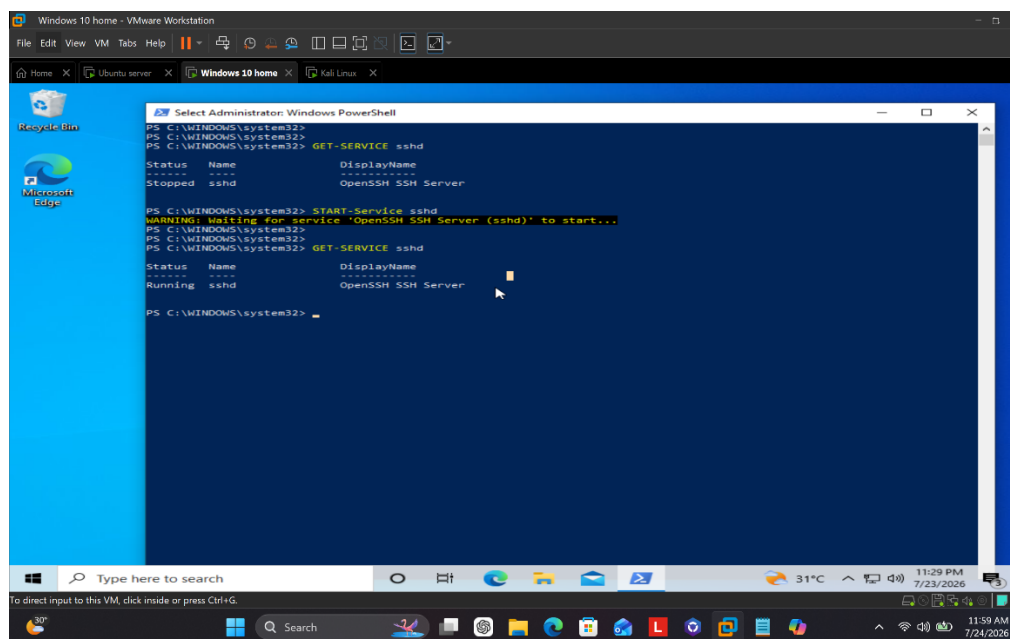


Figure 27: Starting the SSH service

Since Windows use RDP(Remote Desktop Protocol) for remote administration, I had to install SSH as an optional feature, and made sure the services are running using above powershell commands.

6.2 Disabling Windows Security features

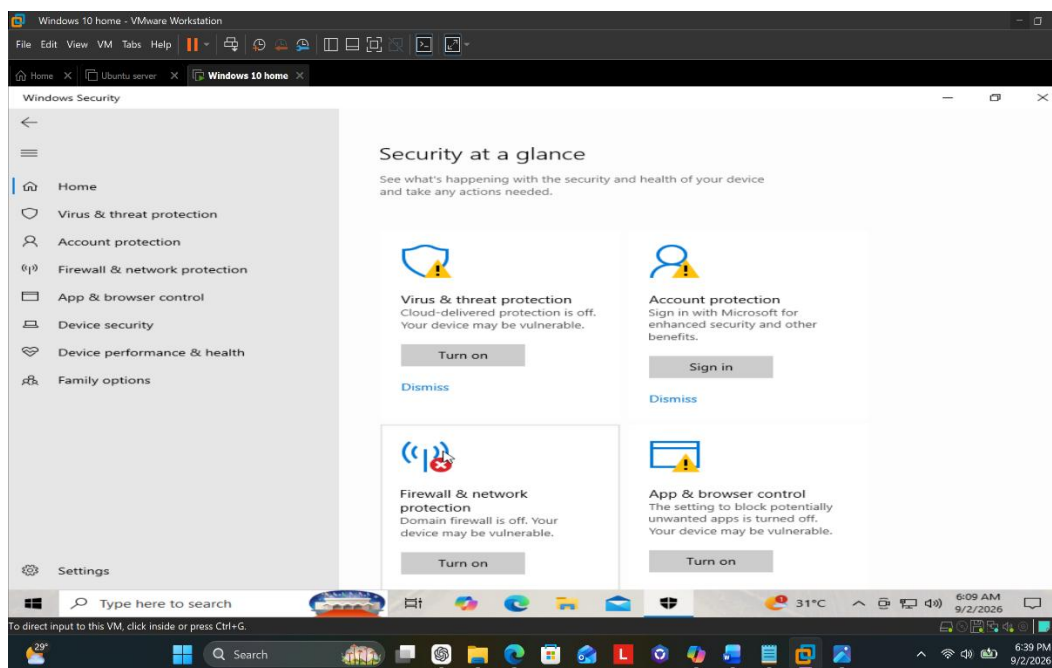


Figure 28: Disabling Windows default Security features

Attack Simulation and Detection

1. Network Enumeration using Nmap (T1046)

1.1 Performing Reconnaissance [TA0043]

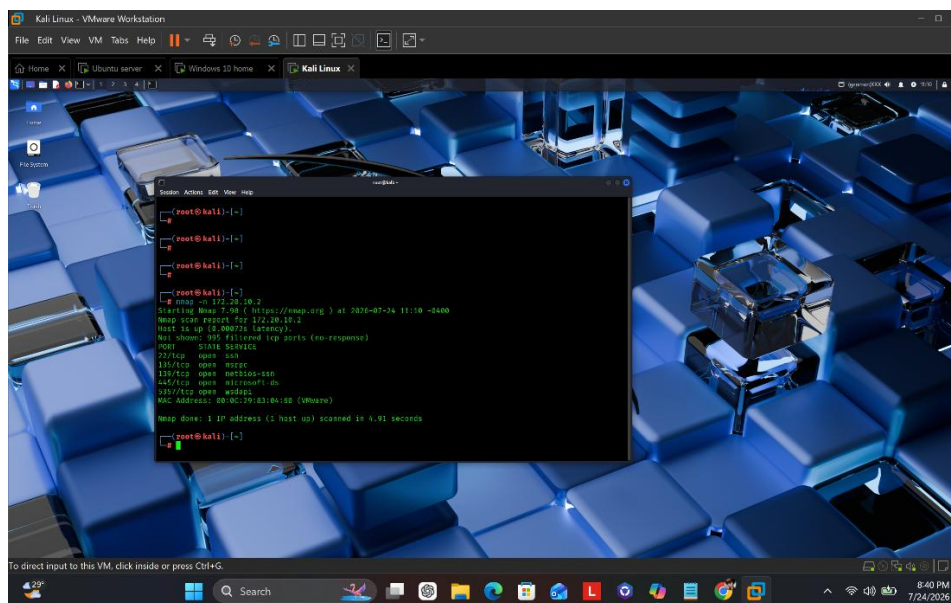


Figure 29: Nmaping to windows machine

Scanned the machines's ip to find open services and as intended ssh port is open.

1.2 Wazuh Detection

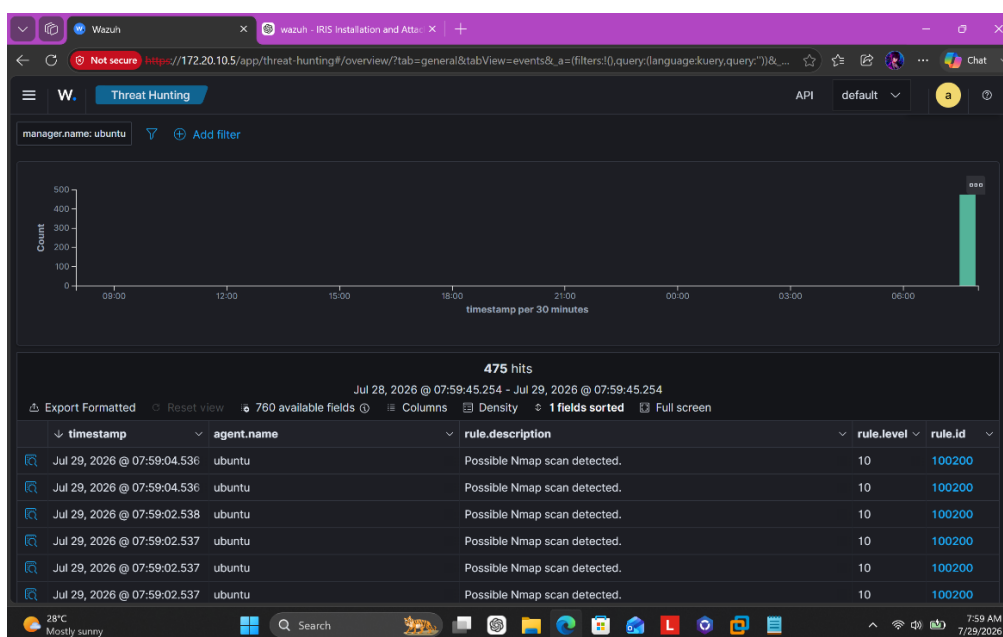


Figure 30: Alert generated for Nmap (1)

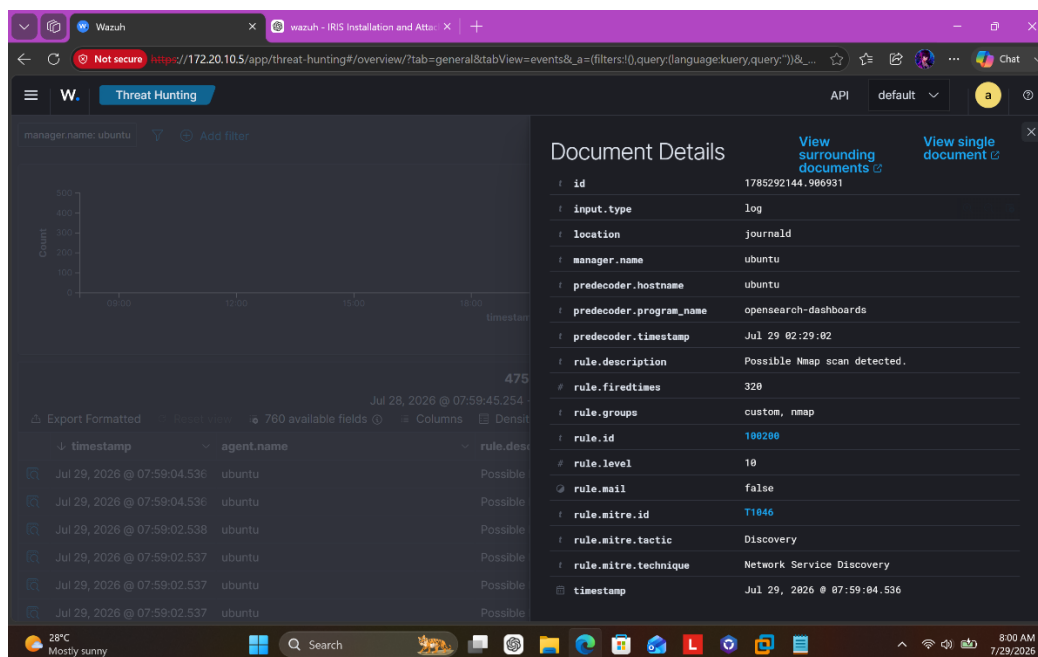


Figure 31: Alert generated for Nmap (2)

A wazuh alert is generated according to our custom rule and got classified into discovery tactic, but this scan was performed pre-compromise of a machine therefore, I am classifying this as a Recon activity.

2. Brute Forcing using Hydra (T1110.001)

2.1 Creating a Password Dictionary

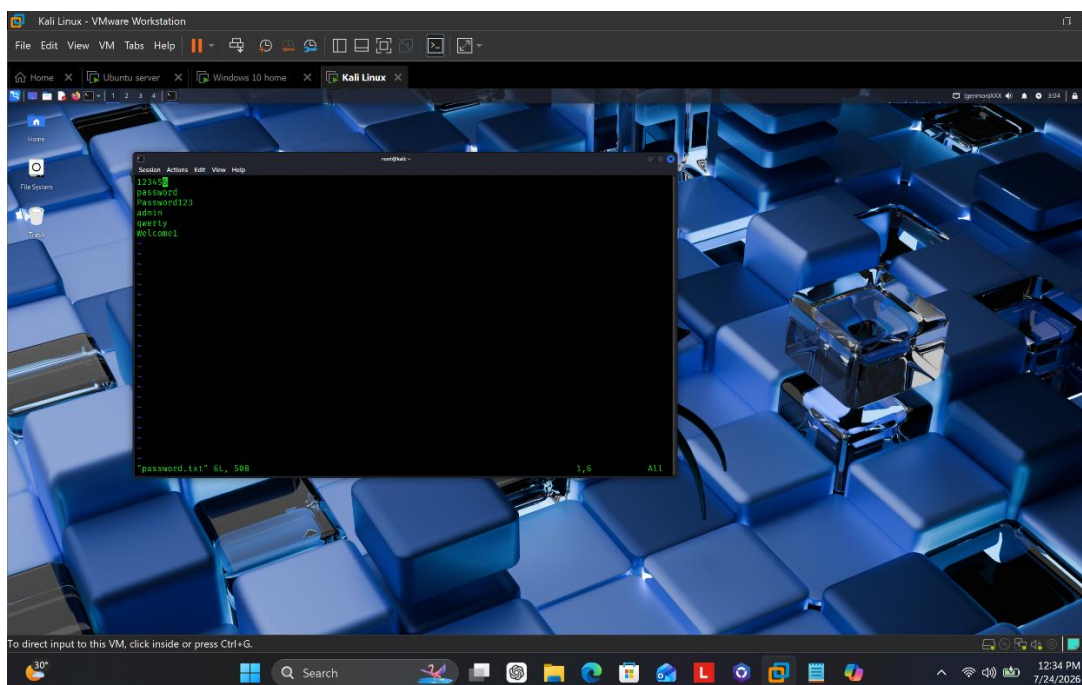


Figure 32: Creating a wordlist

2.2 Credential Guessing (Initial access) [TA0001]

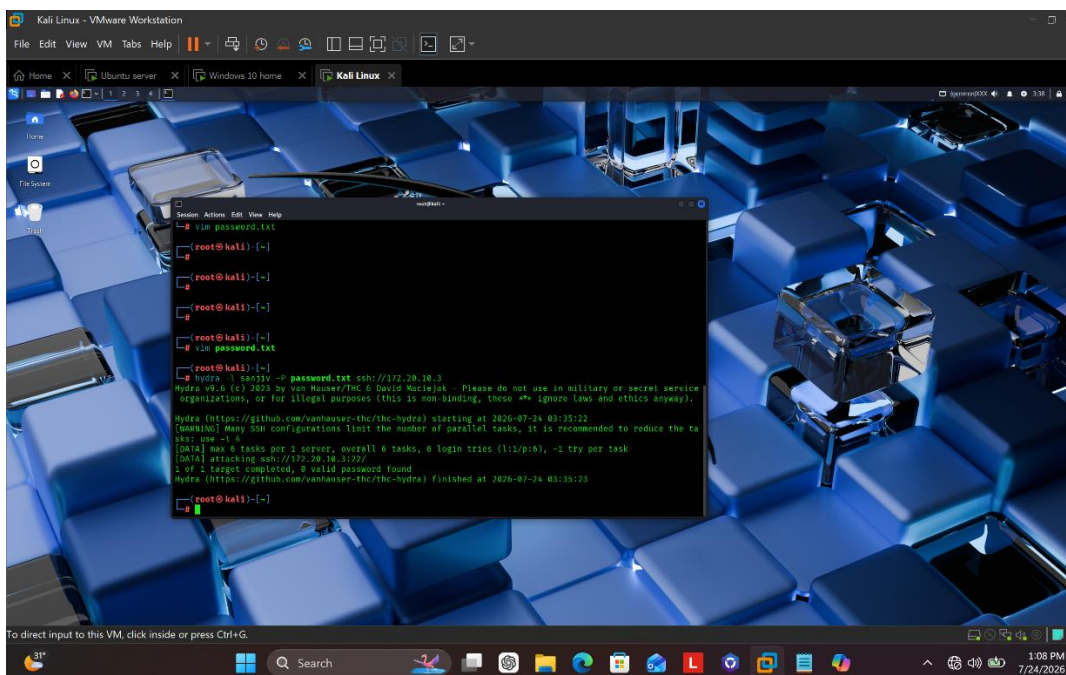


Figure 33: Brute Forcing using Hydra

2.3 Wazuh Detection

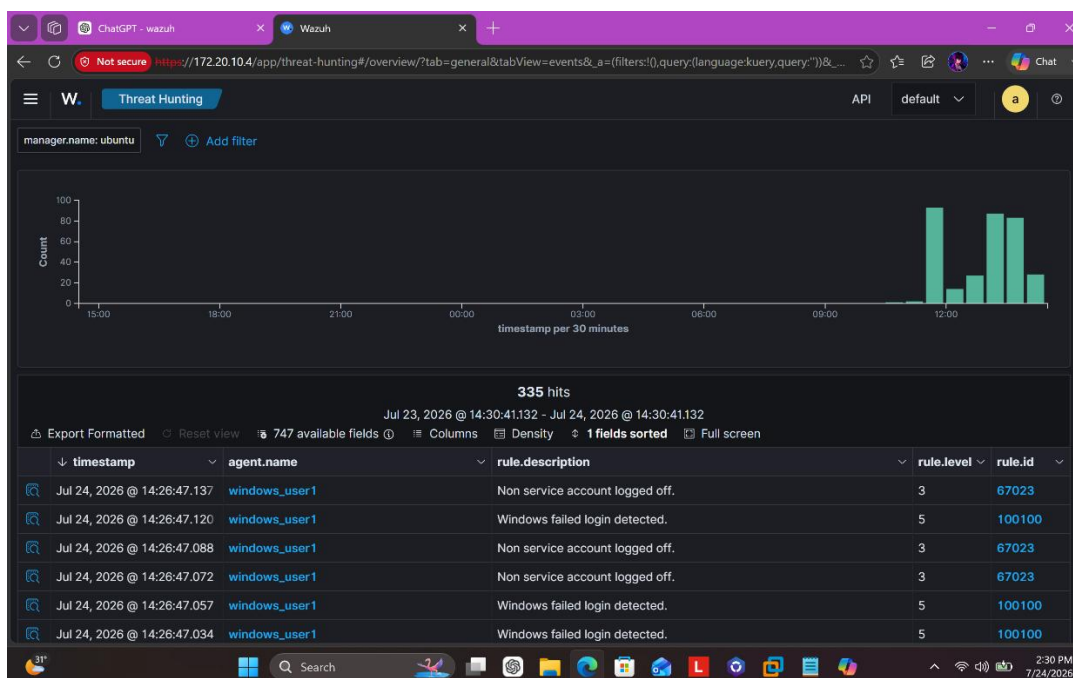


Figure 34: Alert generated for failed SSH

Overall 6 alerts were generated since my password dictionary contained 6 wordlist for network credential brute forcing using Hydra.

3. Establishing a PowerShell reverse shell (T1059.001)

3.1 C2 Listener Setup

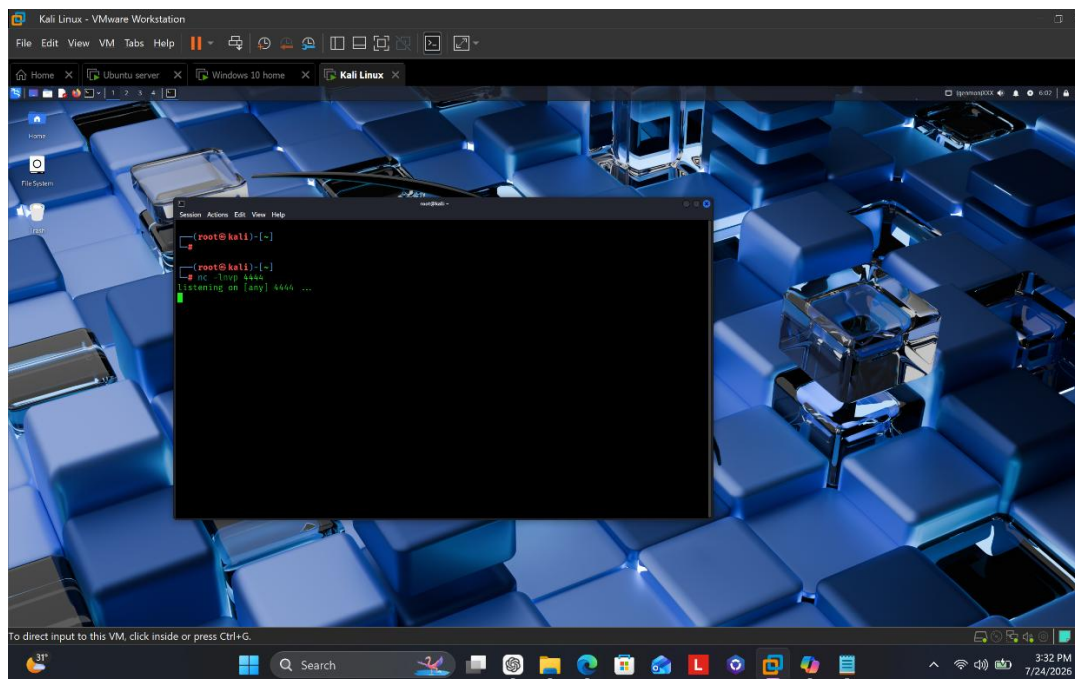


Figure 35: Establishing a listener for the reverse shell

3.2 Executing the PowerShell Reverse shell Payload

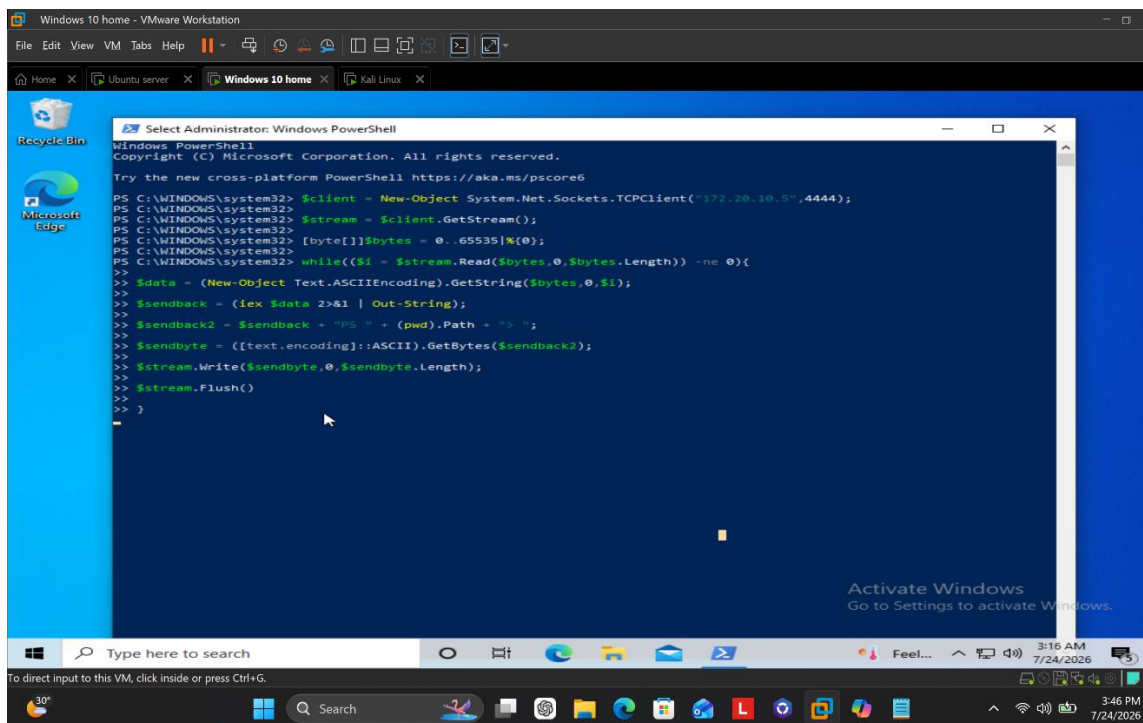


Figure 36: Executing the PowerShell based reverse TCP shell code in victim machine

3.3 Establishing C2(Command and Control) [TA0011]

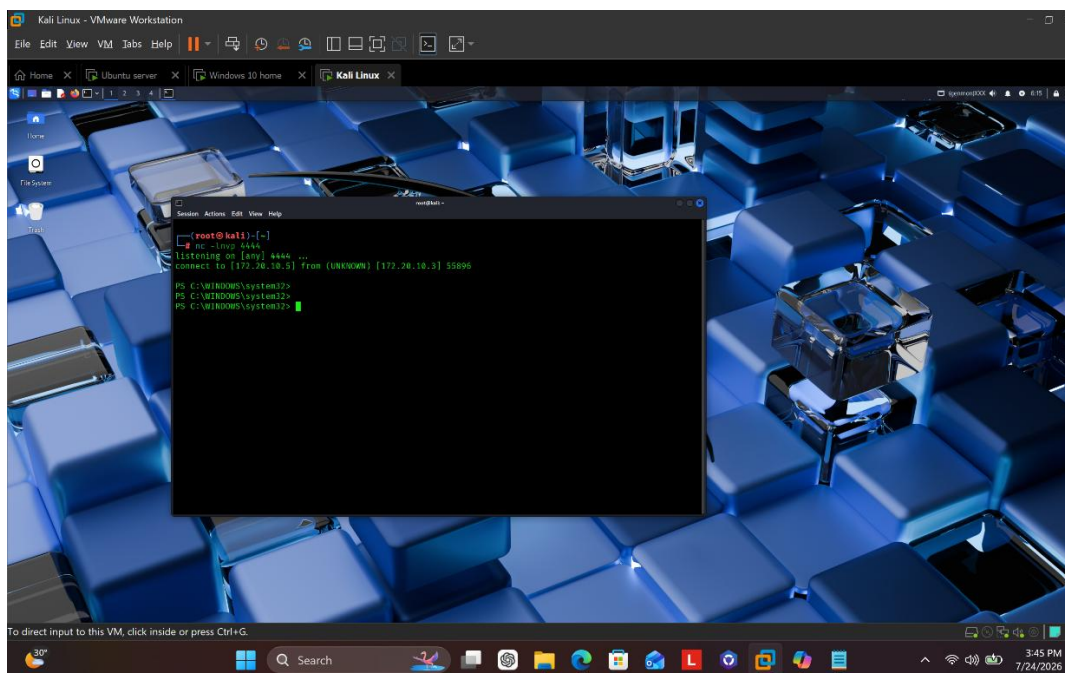


Figure 37: C2 Establishment

In my attacker machine I started a netcat listener on port 4444 and in the victim machine I executed a powershell reverse shell payload which created a TCP connection to the attacker's IP on port 4444, enabling remote command execution.

3.4 Wazuh Detection

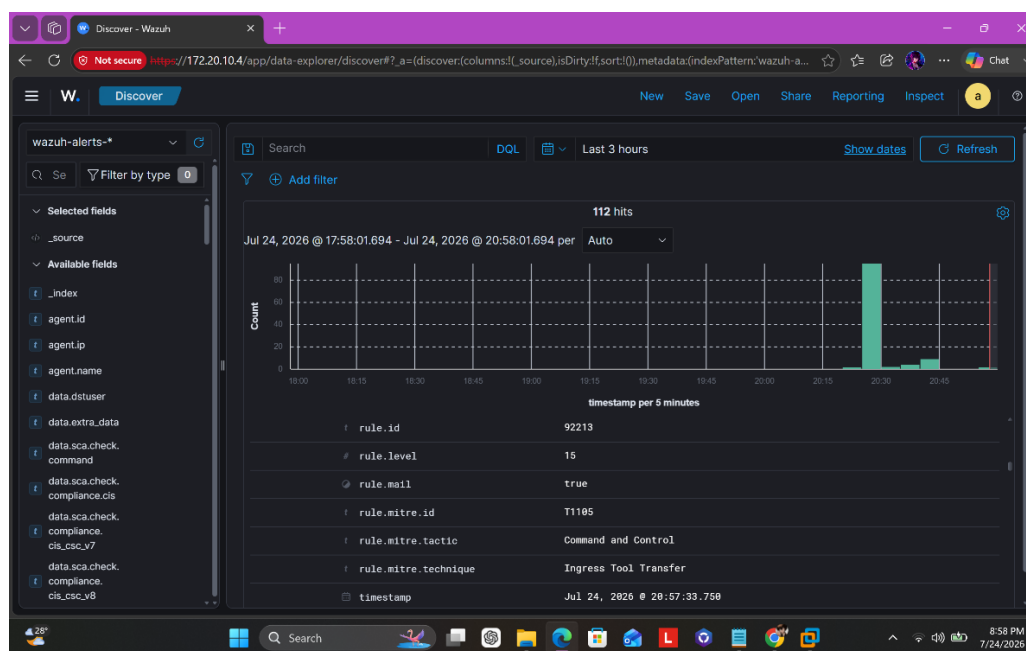


Figure 38: Alert generated for PowerShell Reverse shell

4. File Integrity Violation (T1565.001), (T1222.001)

4.1 Impact Stage of the Attack chain [TA0040]

C:\User1\Sensitive Data\employee_records.txt

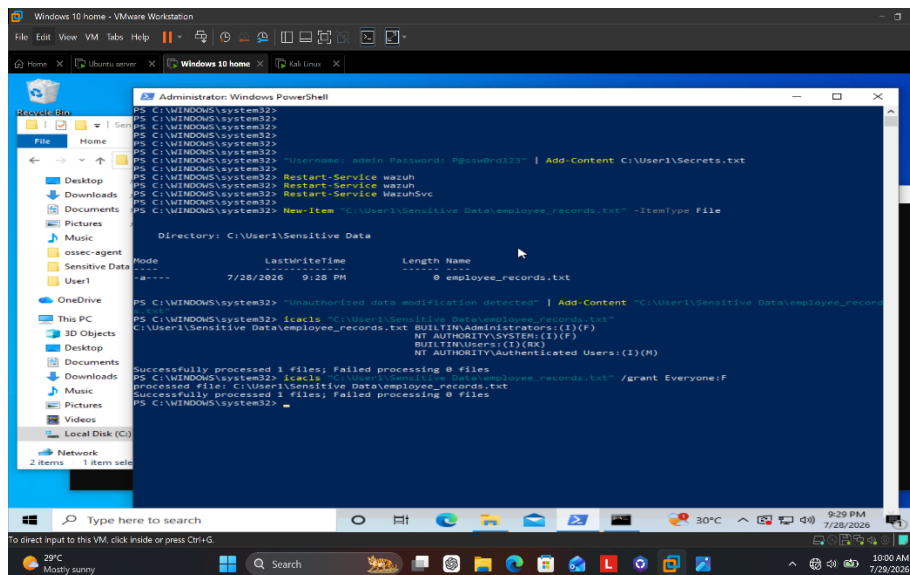


Figure 39: Stored data & permission modification

In this scenario I wanted to trigger a FIM alert in wazuh therefore, I executed a powershell command to create a new file, added content and changed file permission to grant everyone full access over the file and then restarted the wazuh service to make sure the changes are recorded incase it wasn't.

4.2 Wazuh Detection

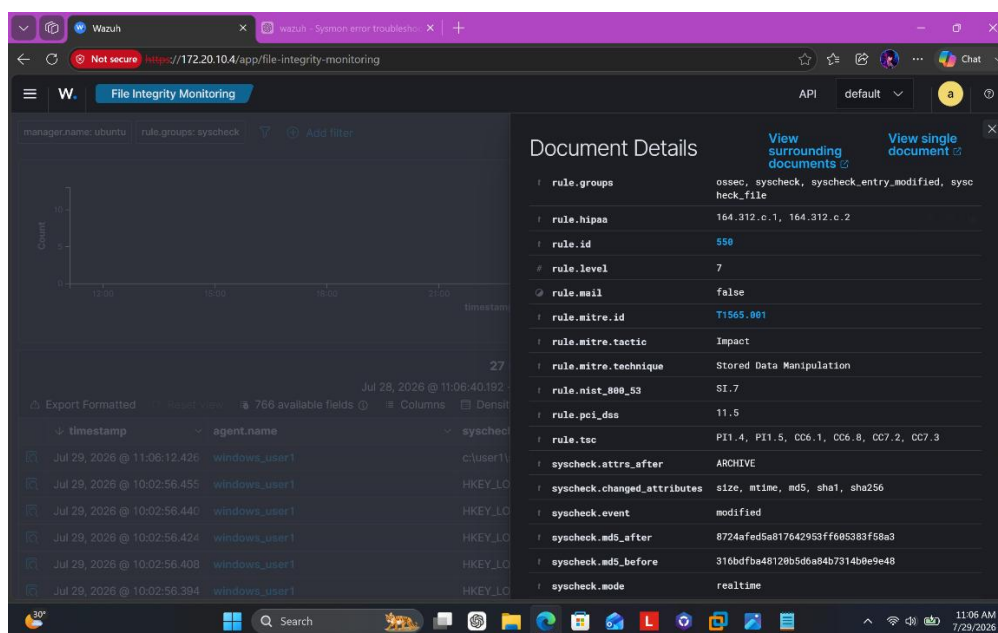


Figure 40: Alert generated for stored data manipulation

Iris Case Creation

1. Case Creation for Hydra Brute Force

The screenshot shows the 'Edit case information' form for a case titled '#2 - Brute Force Attack'. The form includes the following fields:

- Case name:** #2 - Brute Force Attack
- SOC ticket ID:** 110
- Classification:** Intrusion-Attempts: Login attempts
- Owner:** administrator
- State:** In progress
- Outcome:** True Positive with impact
- Customer:** IrisInitialClient
- Reviewer:** administrator
- Severity:** Medium
- Case tags:** A text input field with the placeholder 'Type here' and an 'Add protagonist' button below it.

Figure 41: Creating a case for the Brute Force attack

The screenshot shows the 'Case Timeline' view for the '#2 - Brute Force Attack' case. The timeline displays a single event with the following details:

- Event ID:** 4626
- Timestamp:** 2026-07-24T08:56:45.9390863Z
- Attacker IP:** 172.20.10.5
- Username:** user1
- Hostname:** DESKTOP-55FPLFJ
- Agent name:** windows_user1
- Rule ID:** 001
- Rule level:** 5
- Rule description:** windows failed login detected
- Login type:** 8
- Failure reason:** %%2313
- Status:** 0xc000006d

Figure 42: Adding the extracted information from wazuh

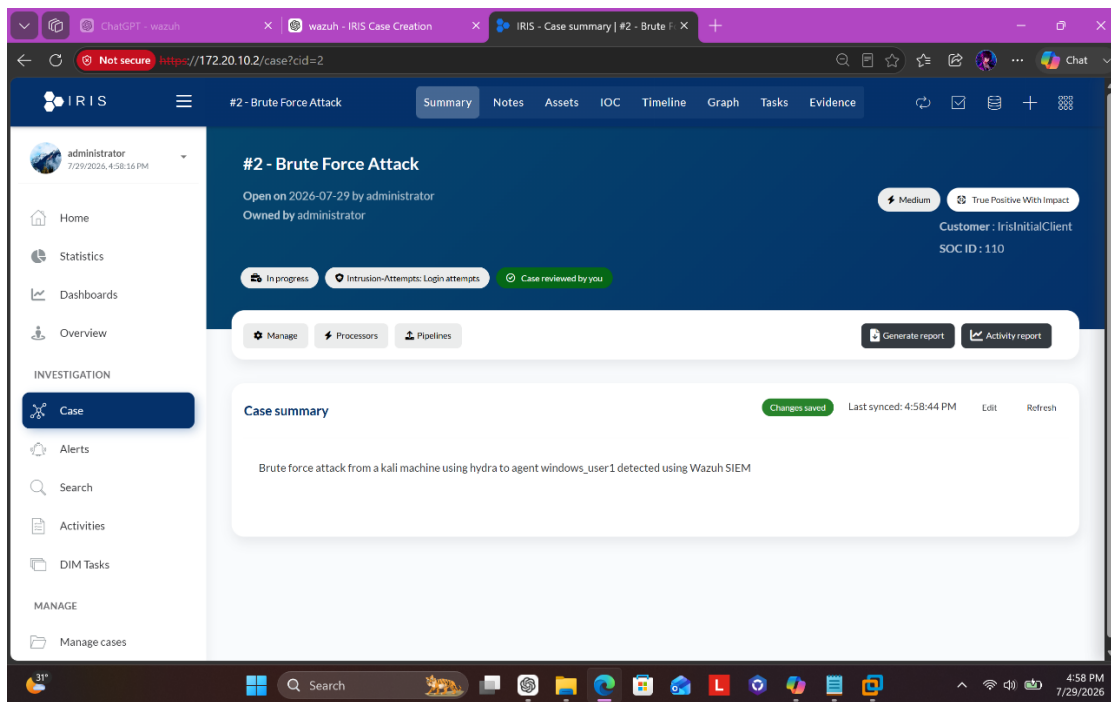


Figure 43: Successfully created a IRIS case for the Brute Force attack

2. Case Creation for PowerShell Reverse shell

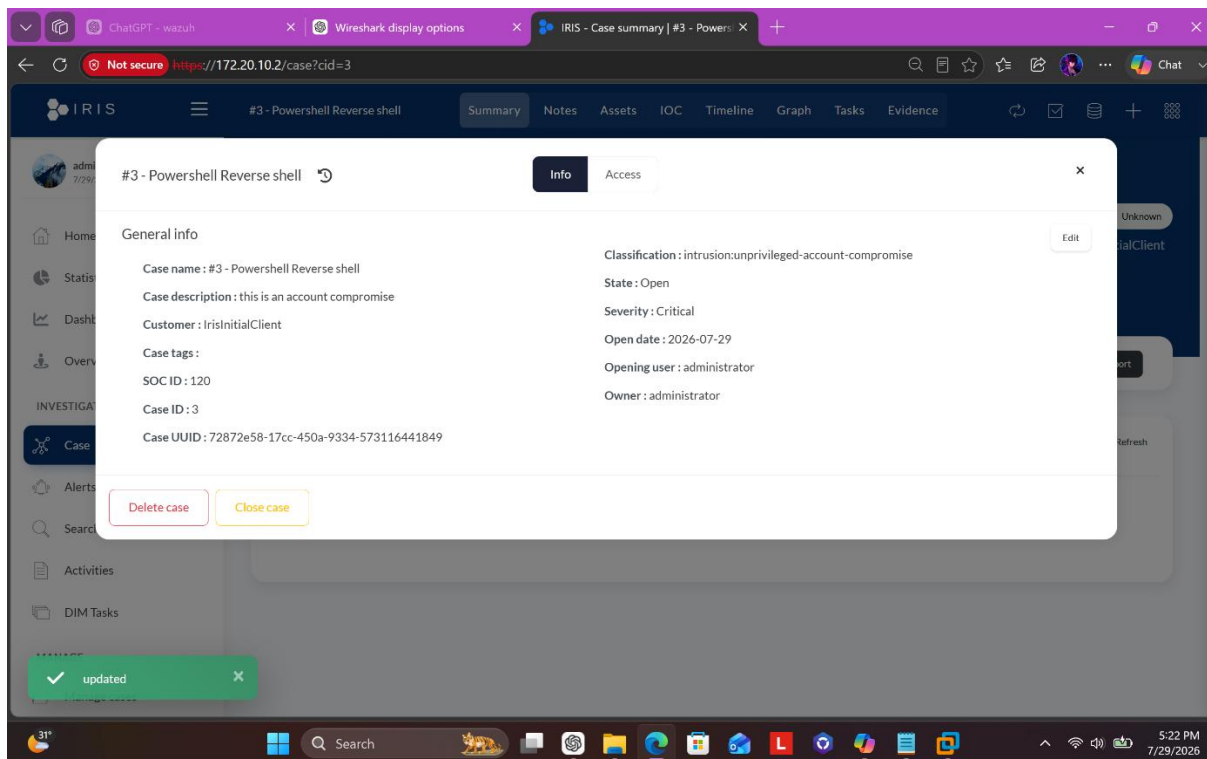


Figure 44: Creating a case for powershell reverse shell

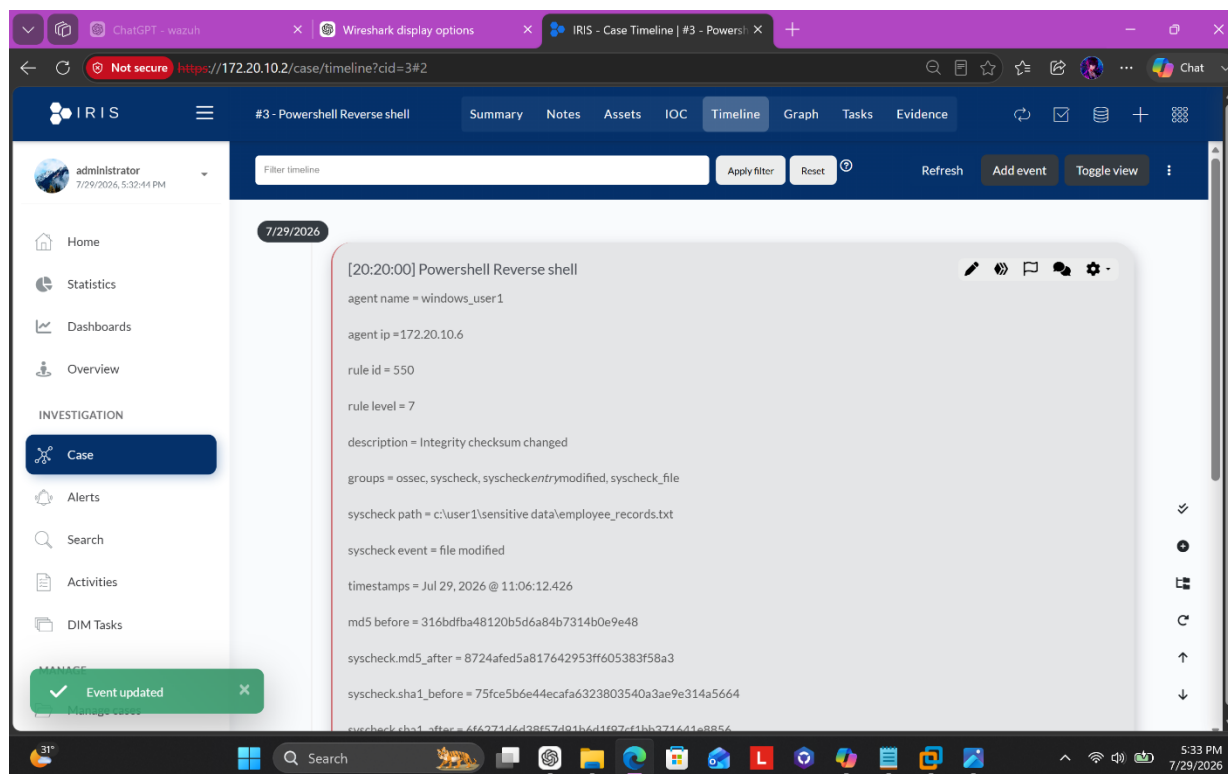


Figure 45: Adding the extracted information from wazuh

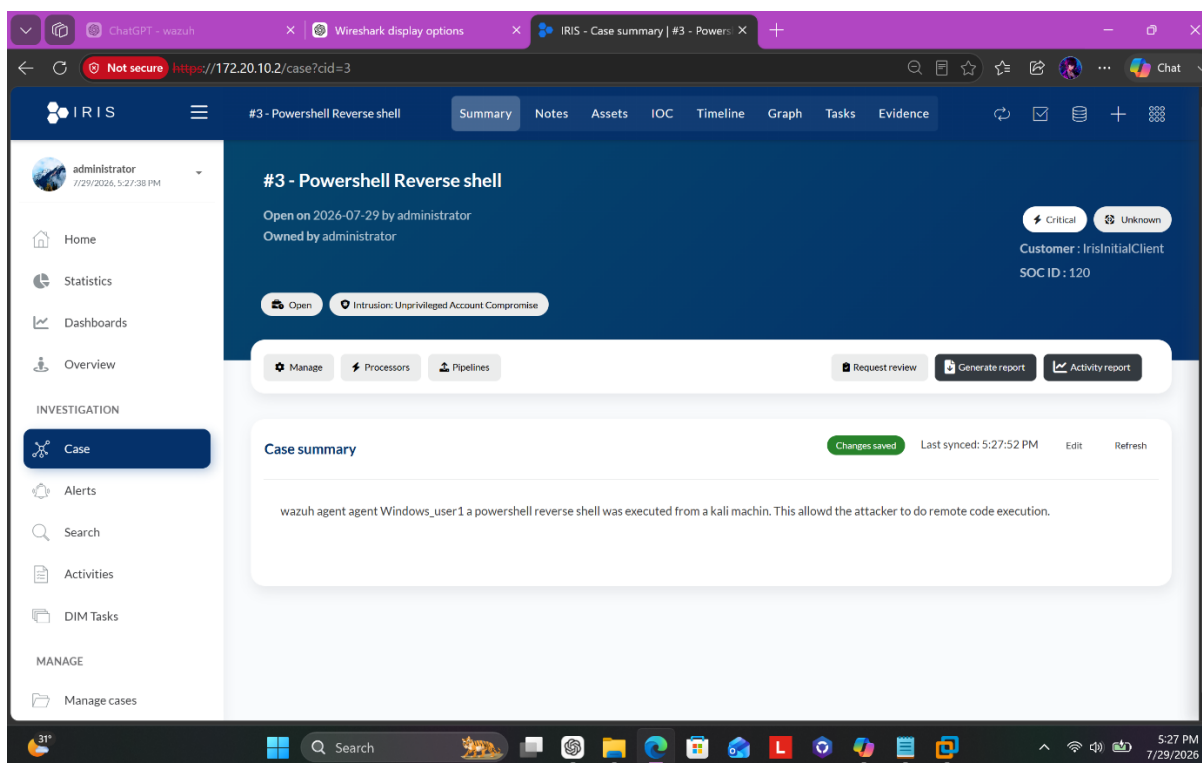


Figure 46: Successfully created a IRIS case for the powershell reverse shell

3. Case Creation for File Integrity Violation

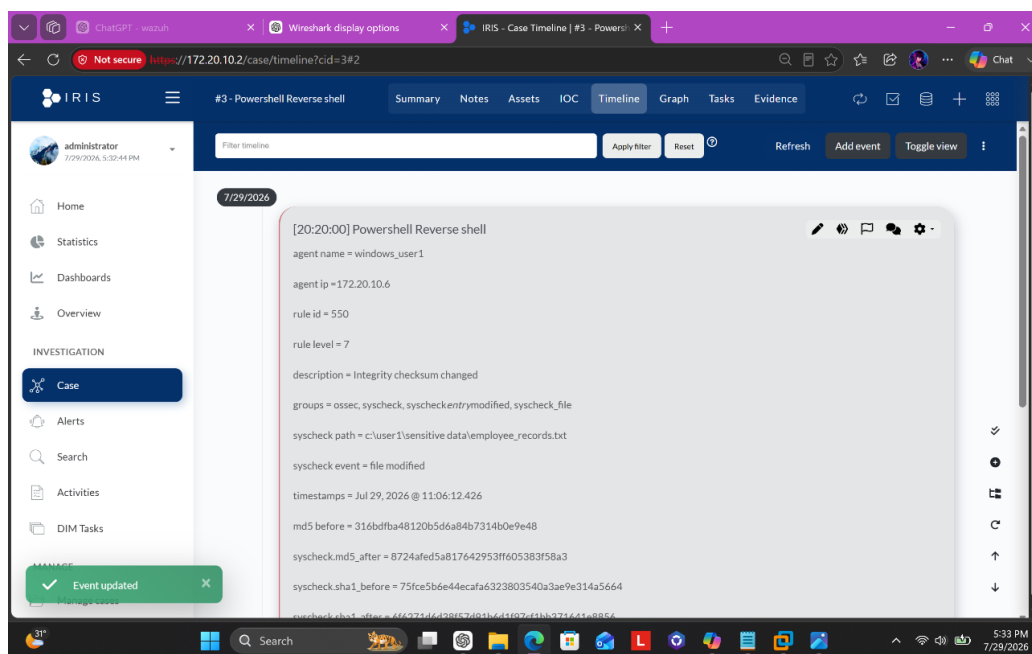


Figure 47: Adding the extracted information from wazuh

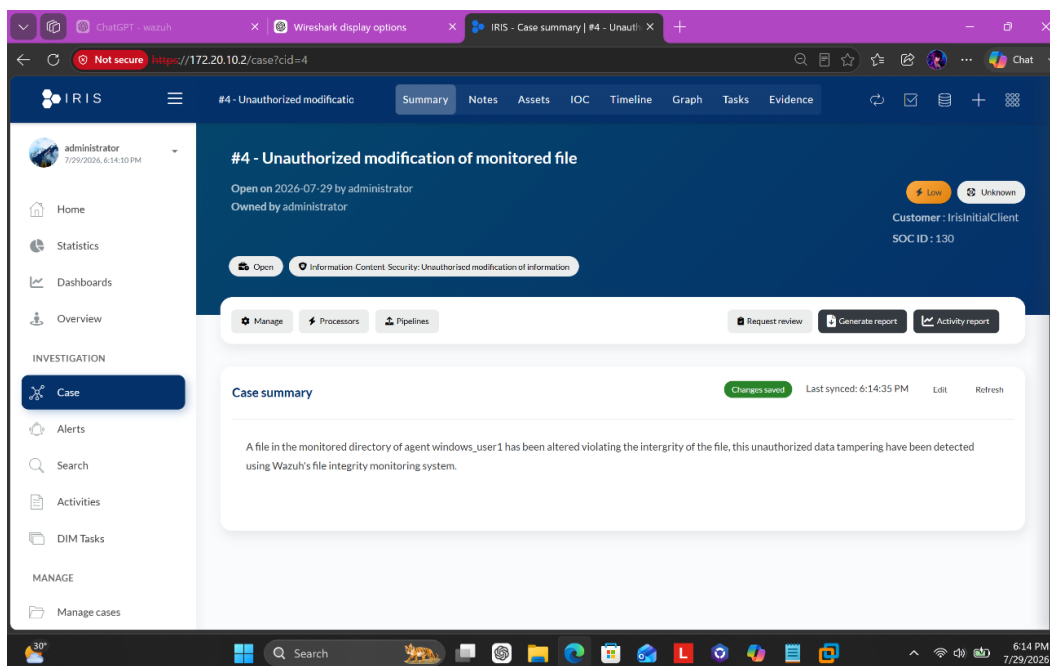


Figure 48: Successfully created a IRIS case for file integrity violation

- NOTE: No case was created for Nmap scan because attacker haven't compromised the machine or didn't attempt to yet, it was just a discovery of hosts, ports and services.

Conclusion

This project helped me gain hands on experience on configuring and implementing industry level tools like wazuh, security monitoring and threat detection platform and IRIS, digital forensics and incident response platform. I also understood the shortcomings of wazuh and how they are resolved like how logs are overwhelming if not refined or saturated using custom detection rules and it also re-triggered my awareness that wazuh does not directly inspect packets instead collect and analyzes network related events utilizing Sysmon to record the activity to windows event log entries and then the wazuh agent reads the windows event log sending the network events to wazuh manager. I also gained fundamental understanding of how attacks like Nmap scan, password guessing using hydra, remote code execution using PowerShell and stored data manipulation are executed and how they are mapped to MITRE ATT&CK framework making it possible to predict attacker behavior and classify alerts for better communication among the security team.